

学习内容

CONTENTS

1

神经元与神经网络

2

BP神经网络

3

卷积神经网络

4

生成对抗网络



5

循环神经网络

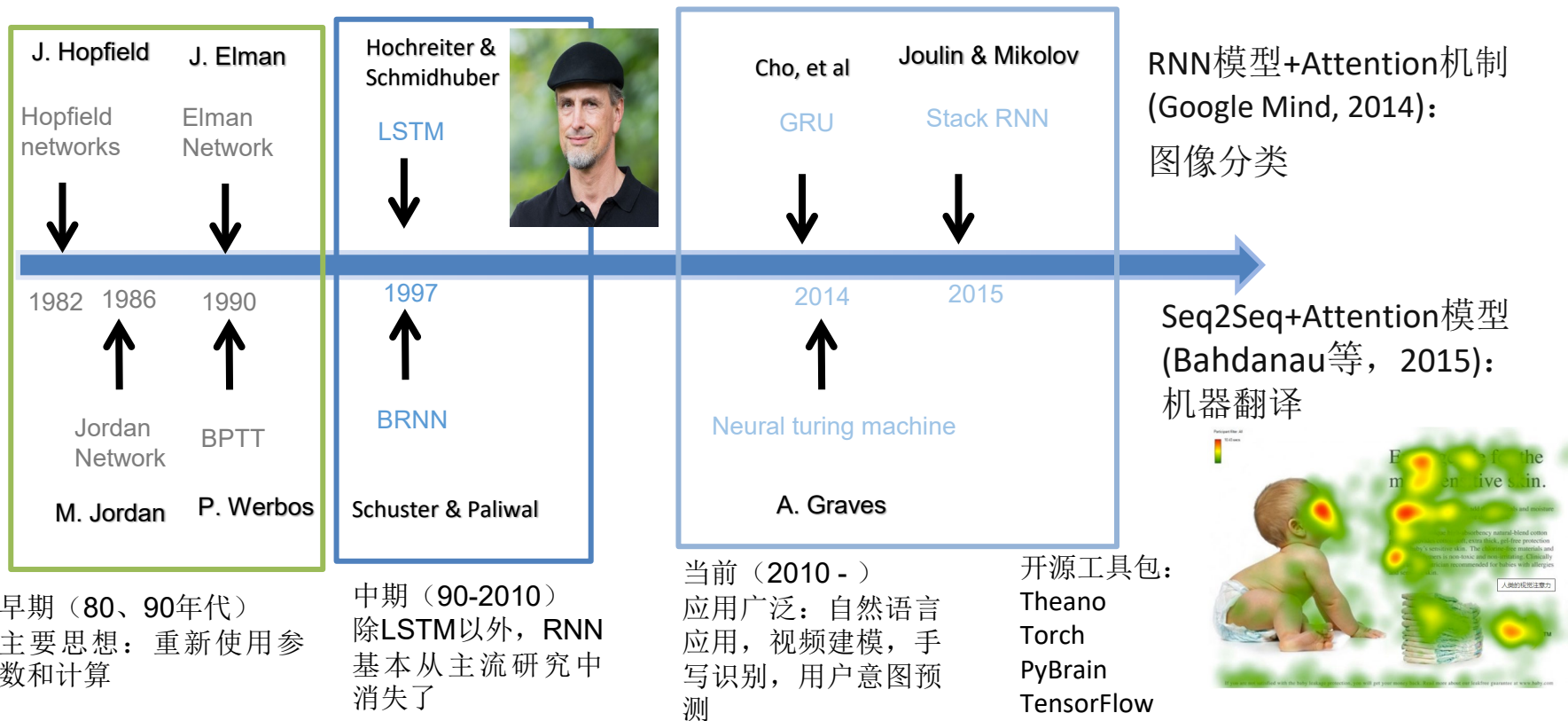
6

Transformer

- 了解循环神经网络的背景、概念
- 掌握循环神经网络的结构
- 熟悉循环神经网络的类别
- 熟悉长短期记忆网络LSTM
- 了解循环神经网络的优缺点、扩展
- 了解循环神经网络的应用（如文本生成、机器翻译、语音识别、生成图像描述、视频标记）



RNN (Recursive Neural Network, 递归神经网络) 发展历史



◆ **循环神经网络 (Recurrent Neural Network , RNN)**”，也叫递归神经网络。这里为了区别与另外一种**递归神经网络 (Recursive Neural Network)**，我们称为**循环神经网络**。



- 循环神经网络 (RNN, Recurrent Neural Network)

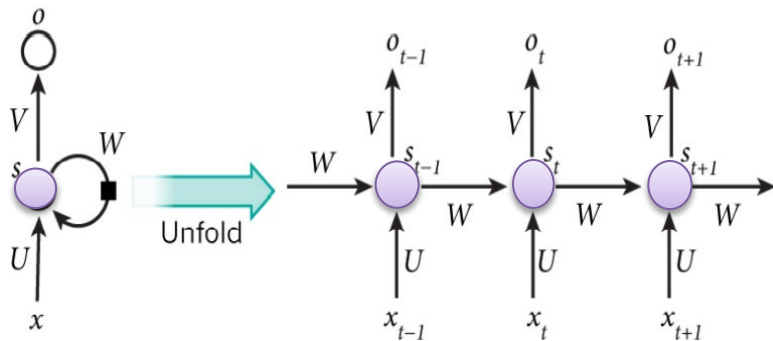
- RNN基本原理

- 长短期记忆网络

- (Long Short-Term Memory, LSTM)

- 门限循环单元

- (Gated Recurrent Unit, GRU)



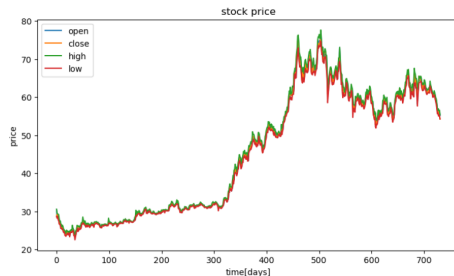
- RNN不同于前向神经网络，它的层内、层与层之间的信息可以双向传递，更高效地存储信息，利用更复杂的方法来更新规则，通常用于处理信息序列的任务。

✓ 循环神经网络 (Recurrent Neural Network, RNN) 是一种对**序列数据**建模的神经网络。

• 序列数据，例如

- 文本，是字母和词汇的序列；
- 语音，是音节的序列；
- 视频，是图像的序列；
- 气象观测数据；
- 股票交易数据等等

• 具有先后顺序的数据一般叫作序列(Sequence)



•主要应用领域:

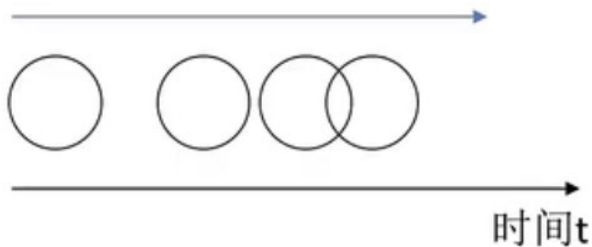
- 语音识别
- 音乐合成
- 情感分类
- DNA序列分析
- 机器翻译
- 视频标记
- 命名实体识别



•核心思想:

- 数据间存在顺序关系，每个数据和它之前的数据存在关联。
- 通过神经网络在时序上的展开，我们能够找到数据之间的序列相关性。

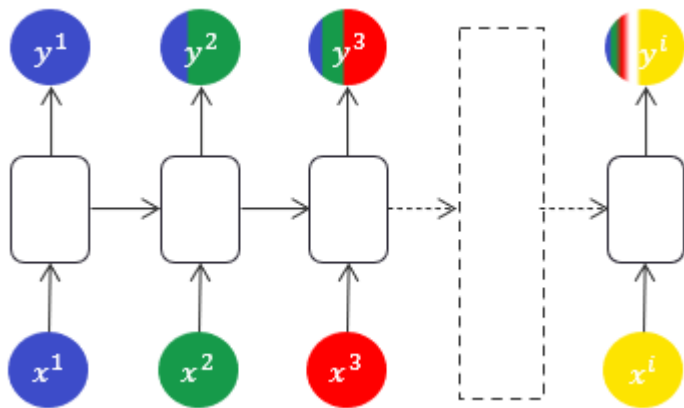
序列数据：数据元素间具有顺序关系



“不吃饭” VS “吃饭不”

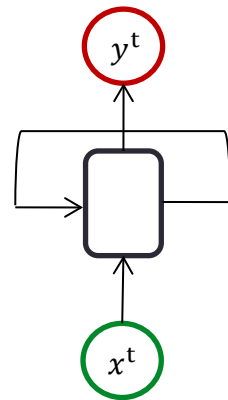
文本数据
怎么处理？

把前面序列信息提取部分，用于后面预测



沿时间轴展开的RNN模型

=



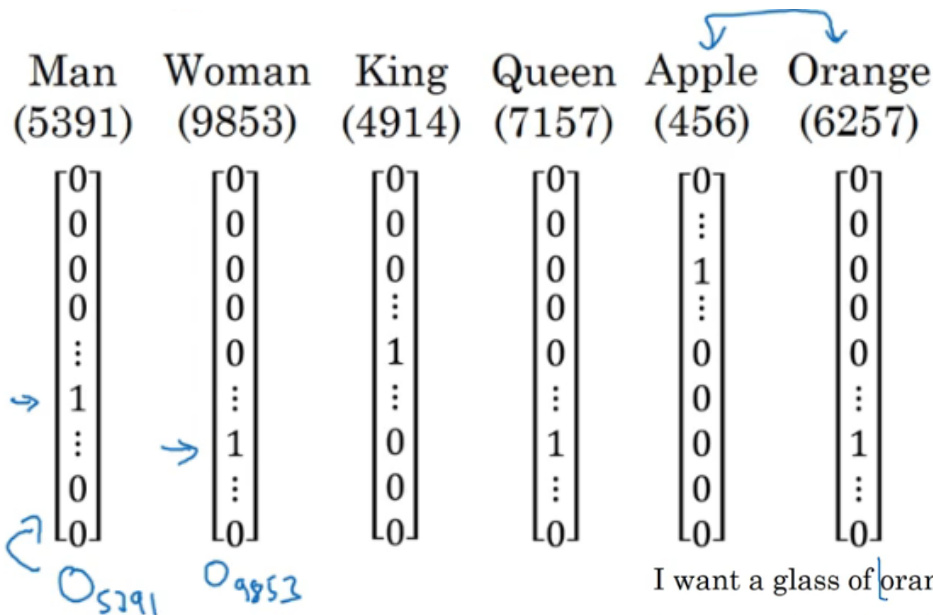
折叠的RNN模型

把文字编码为数值的过程叫作词嵌入（**Word Embedding**），也称词向量。

建立一个词汇-数值一一对应的字典，然后把输入词汇转化为数值矩阵。

One-hot（独热） 的编码方式实现 Word Embedding 简单直观，编码过程不需要学习和训练，但会导致维数灾，向量之间独立，看不出无关联。

$$\left\{ \begin{array}{lcl} a & : & 1 \\ am & : & 2 \\ \dots & : & \dots \\ the & : & 100 \\ \dots & : & \dots \\ courses & : & 1000 \\ are & : & 1001 \\ flare & : & 1002 \\ \dots & : & \dots \end{array} \right\}$$



I want a glass of orange juice.

I want a glass of apple ?

8.5 词向量



$$\mathbf{e}_{\text{Apple}} \approx \mathbf{e}_{\text{Orange}}$$

$$|V|=10000$$

$$\mathbf{e}_{\text{Man}} - \mathbf{e}_{\text{Woman}} \approx \mathbf{e}_{\text{King}} - \mathbf{e}_{\text{Queen}}$$

$$\begin{bmatrix} -2 \\ -0.01 \\ 0.01 \\ 0.08 \\ \vdots \end{bmatrix} \approx \begin{bmatrix} -1.92 \\ -0.02 \\ 0.01 \\ 0.01 \\ \vdots \end{bmatrix}$$

Featurized representation: word embedding

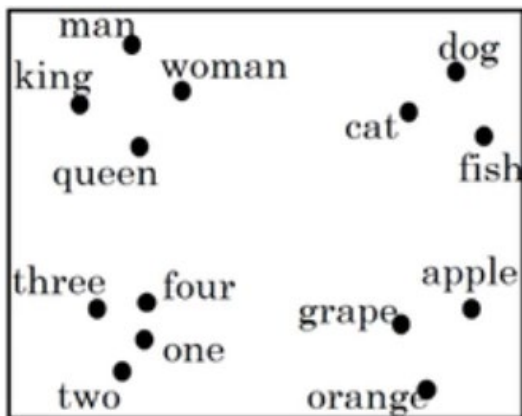
	Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
Gender	-1	1	-0.95	0.97	0.00	0.01
Royal	0.01	0.02	<u>0.93</u>	<u>0.95</u>	-0.01	0.00
Age	0.03	0.02	0.7	0.69	0.03	-0.02
Food	0.04	0.01	0.02	0.01	0.95	0.97
size	⋮	⋮				
cost						
alive						
verb						

$\mathbf{e}_{5391}$ $\mathbf{e}_{9853}$

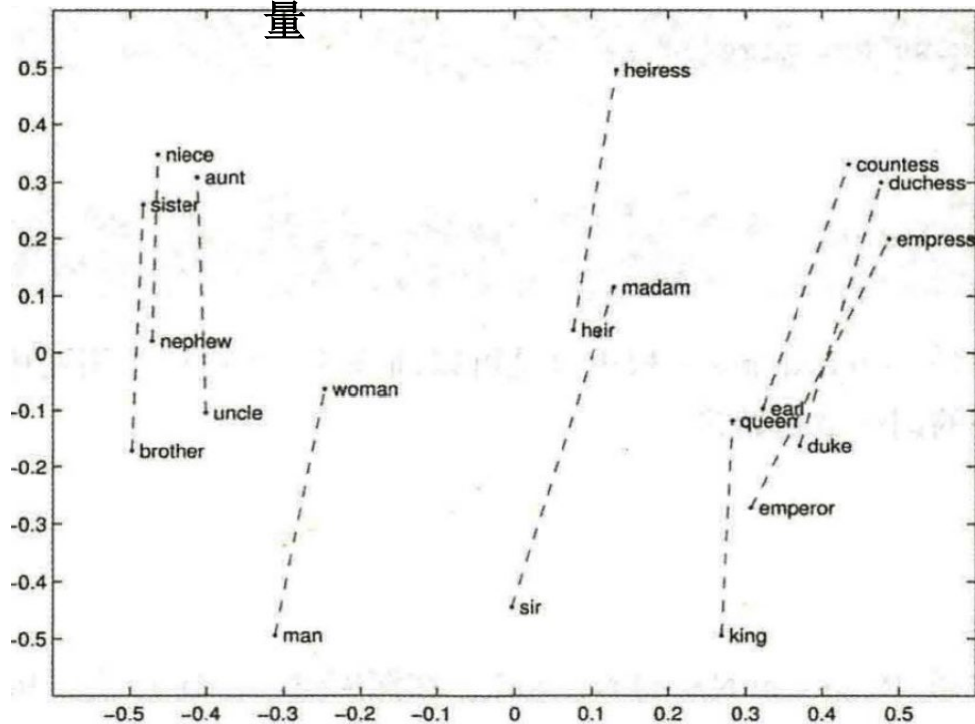
I want a glass of orange _____.
 I want a glass of apple _____.

词向量（词嵌入）的特性：

- 1) 词向量中语义上相近的词距离相近；
- 2) 词向量中语法上相近的词距离相近。

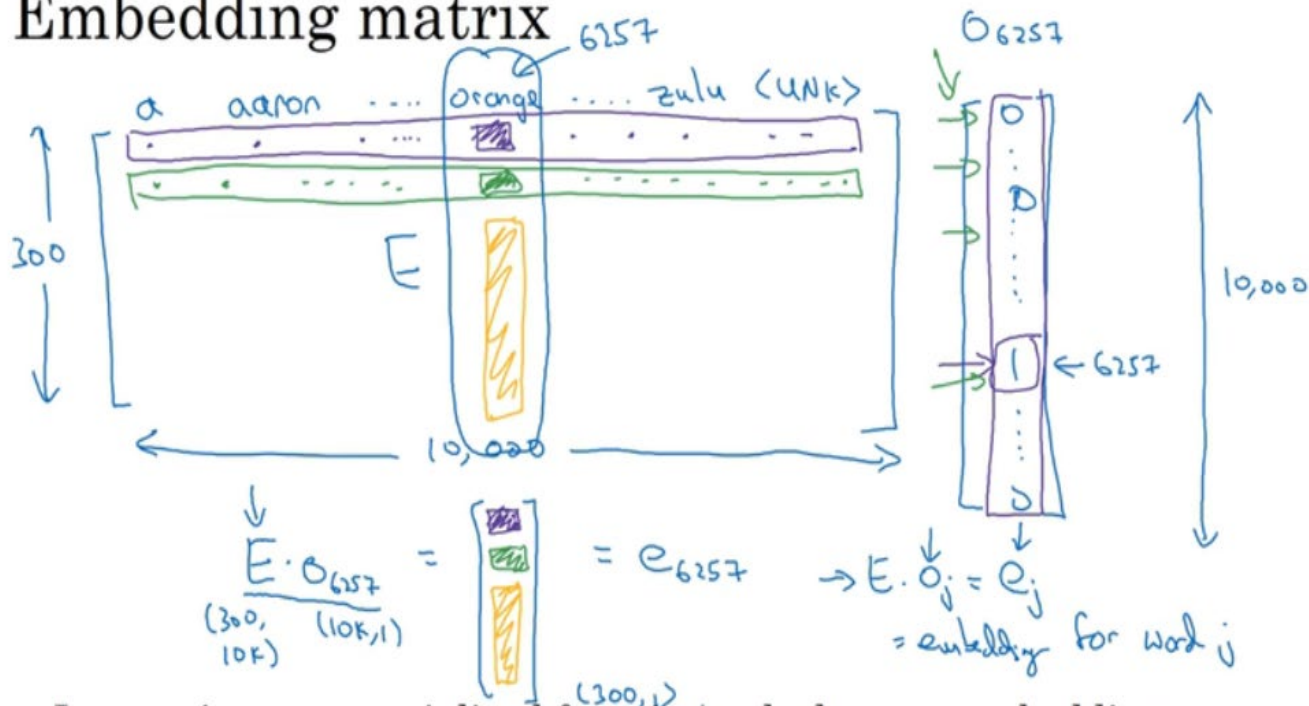


男性词和女性词对应的词向量



词向量的可视化表示（t-SNE）

Embedding matrix



Word Vector
Lookup Table!

300 features

10,000 words

In practice, use specialized function to look up an embedding.

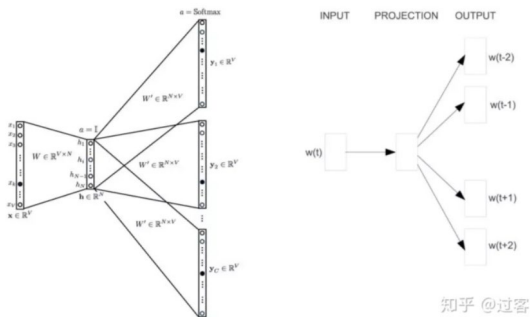
Andrew Ng

预训练的词向量

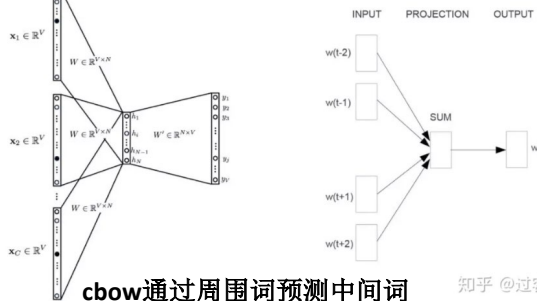
100+ Chinese Word Vectors 上百种预训练中文词向量：
github.com/Embedding/Chinese-Word-Vectors

预训练语言模型（Pre-trained Language Models, PLMs）：**word2vec**、**glove**、**fasttext**、**ELMo**、**BERT**、**GPT**等。

Word2vec (Google, 2013)

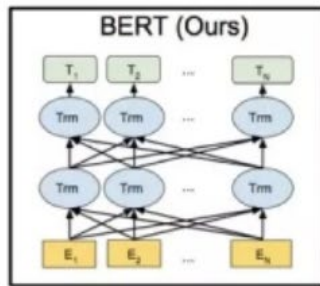


skipgram通过中间词预测周围词

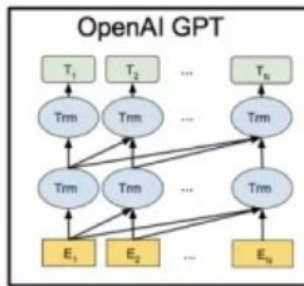


cbow通过周围词预测中间词

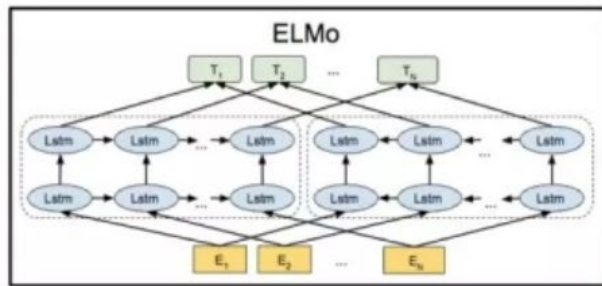
- glove** (2014)：矩阵分解式词向量预训练模型，目标词 w 的Embedding表示取决于同语境中的词 c 的共现关系
- glove**矩阵分解是基于全局语料的，而**word2vec**是基于部分语料训练的
- glove**和**word2vec**、**fasttext**(2017)一样词向量都是静态的，无法解决多义词问题



2018.10



2018.6



2018.3

Figure 1: Differences in pre-training model architectures. BERT uses a bidirectional Transformer. OpenAI GPT uses a left-to-right Transformer. ELMo uses the concatenation of independently trained left-to-right and right-to-left LSTM to generate features for downstream tasks. Among three, only BERT representations are jointly conditioned on both left and right context in all layers.

NLP中的十个预训练模型https://blog.csdn.net/weixin_39602569/article/details/111624689
源码+数据Github网址: <https://github.com/zlsdu/Word-Embedding>

Embedding层

- **Embedding** 的权重（参数）可以训练，既可以随机初始化，也可以采用预训练好的词向量初始化。

nn.Embedding

```
tensor([[0, 1],
        [2, 3],
        [4, 5]])
torch.Size([3, 2, 2])
tensor([[[[-0.5461, -0.1320],
          [ 0.5699, -0.8629]],

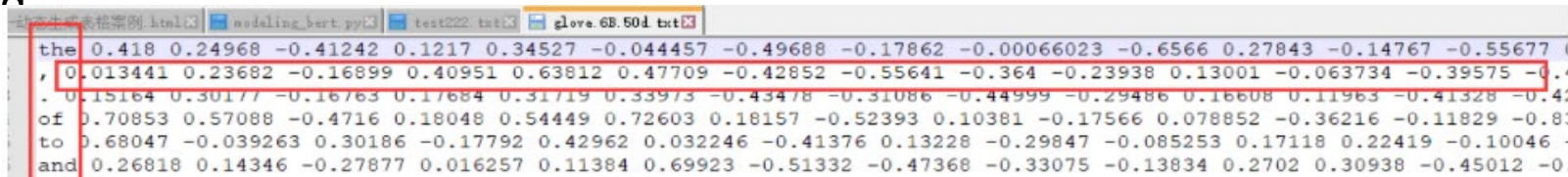
         [[-0.4464, -0.7228],
          [ 0.4974,  1.1535]],

         [[-0.6861, -0.6191],
          [-0.4399,  0.0324]]], grad_fn=<EmbeddingBackward>])
torch.Size([10, 2])
```

预训练的词向量

早期的预训练模型：**Word2Vec**和**GloVe**，已经在海量语料库训练得到了较好的词向量表示，可以直接导出学习到的词向量表，迁移到其它任务。

例如 **glove.6B.50d.txt**，词汇量为 **40** 万，每个单词使用长度为 **50** 的向量表示，只需要下载对应的模型文件（“glove.6B.zip”约842MB）即可。其他还有100d、200d和300d



```
the 0.418 0.24968 -0.41242 0.1217 0.34527 -0.044457 -0.49688 -0.17862 -0.00066023 -0.6566 0.27843 -0.14767 -0.55677  
0.013441 0.23682 -0.16899 0.40951 0.63812 0.47709 -0.42852 -0.55641 -0.364 -0.23938 0.13001 -0.063734 -0.39575  
0.15164 0.30177 -0.16763 0.17684 0.31719 0.33973 -0.43478 -0.31086 -0.44999 -0.29486 0.16608 0.11963 -0.41328  
of 0.70853 0.57088 -0.4716 0.18048 0.54449 0.72603 0.18157 -0.52393 0.10381 -0.17566 0.078852 -0.36216 -0.11829  
to 0.68047 -0.039263 0.30186 -0.17792 0.42962 0.032246 -0.41376 0.13228 -0.29847 -0.085253 0.17118 0.22419 -0.10046  
and 0.26818 0.14346 -0.27877 0.016257 0.11384 0.69923 -0.51332 -0.47368 -0.33075 -0.13834 0.2702 0.30938 -0.45012
```

```
import torchtext.vocab as vocab  
# print(vocab.pretrained_aliases.keys())  
# print([key for key in vocab.pretrained_aliases.keys() if "glove" in key])  
cache_dir = "./.vector_cache/"  
# glove = vocab.pretrained_aliases["glove.6B.50d"](cache=cache_dir)  
glove = vocab.GloVe(name='6B', dim=50, cache=cache_dir) # 与上面等价  
print("一共包含%d个词。" % len(glove.stoi))  
print(glove.stoi['beautiful'], glove.itos[3366]) # (3366, 'beautiful')  
print(glove['beautiful'])
```

一共包含400000个词。

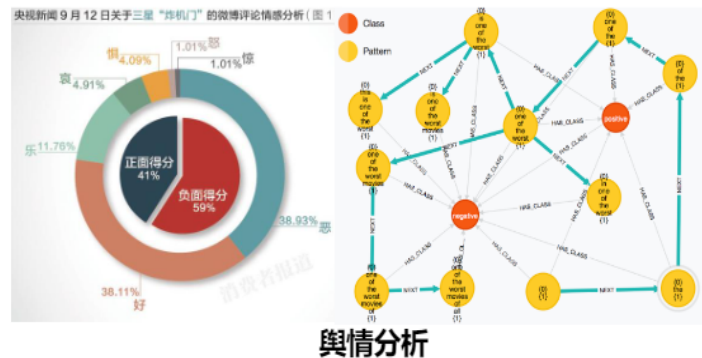
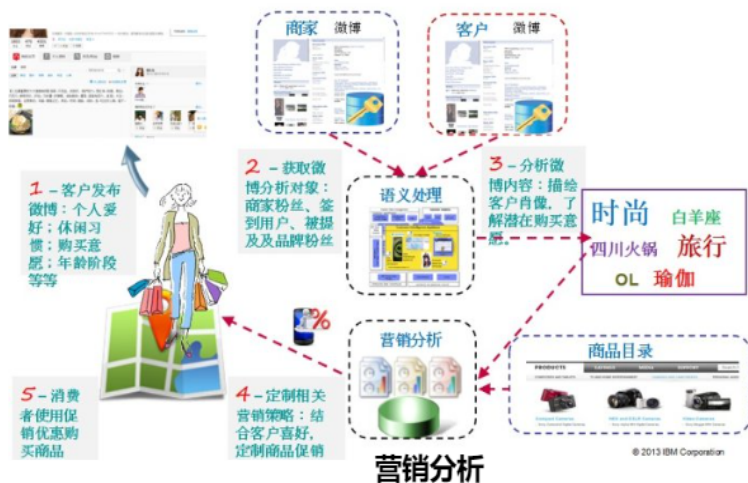
3366 beautiful

```
tensor([ 0.5462, 1.2042, -1.1288, -0.1325, 0.9553, 0.0405, -0.4786, -0.3397,  
-0.2806, 0.7176, -0.5369, -0.0046, 0.7322, 0.1210, 0.2809, -0.0881,  
0.5973, 0.5526, 0.0566, -0.5025, -0.6320, 1.1439, -0.3105, 0.1263,  
1.3155, -0.5244, -1.5041, 1.1580, 0.6880, -0.8505, 2.3236, -0.4179,  
0.4452, -0.0192, 0.2897, 0.5326, -0.0230, 0.5896, -0.7240, -0.8522,  
-0.1776, 0.1443, 0.4066, -0.5200, 0.0908, 0.0830, -0.0220, -1.6214,  
0.3458, -0.0109])
```

循环神经网络

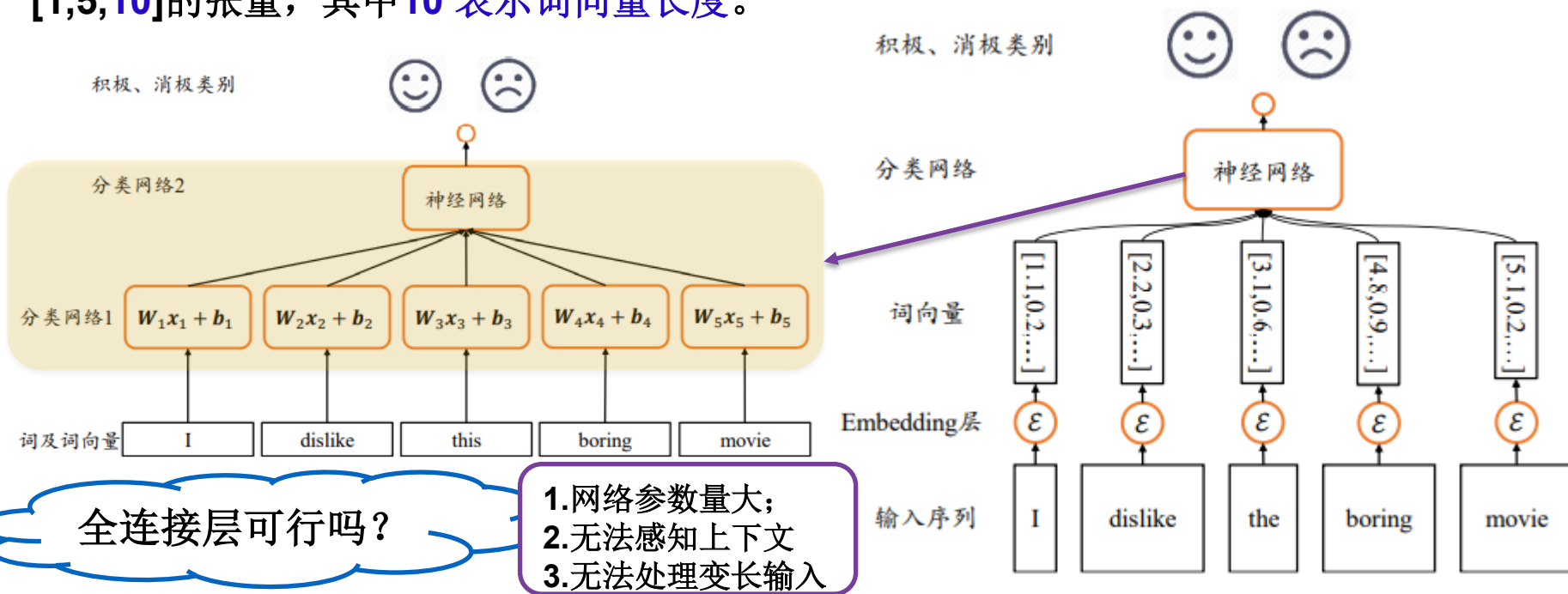
情感分析(sentiment analysis)是指根据文本所表达的含义和情感信息将文本划分成褒义、贬义或多种类型，是对文本作者倾向性和观点、态度的划分，有时也称倾向性分析 (opinion analysis)。

按粒度可分为词汇级、句子级和篇章级的情感分析，核心任务主要包含观点性及倾向性识别、观点要素抽取等任务。



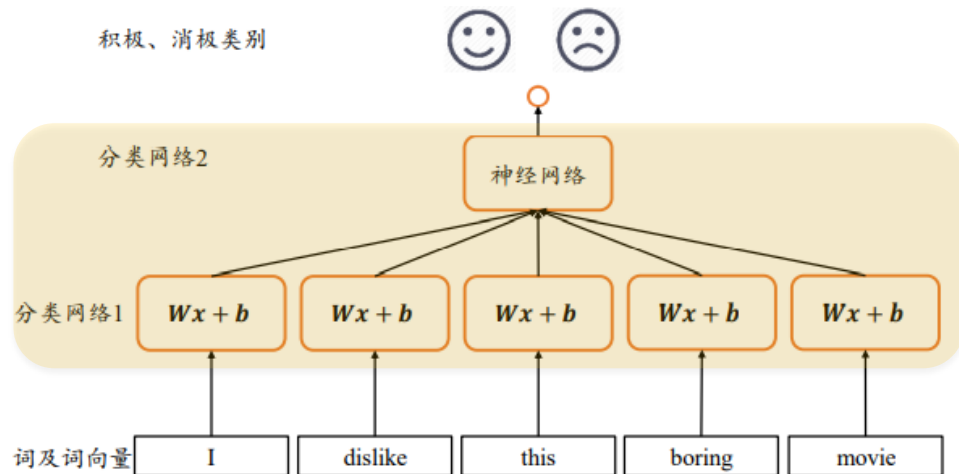
情感分类任务：分析给出的文本序列，提炼出文本数据表达的整体语义特征，从而预测输入文本的情感类型：**正面评价**或者**负面评价**。

例如：“**I dislike this boring movie**”，通过 **Embedding** 层，可以转换为 **shape**为 **[1,5,10]**的张量，其中**10**表示词向量长度。



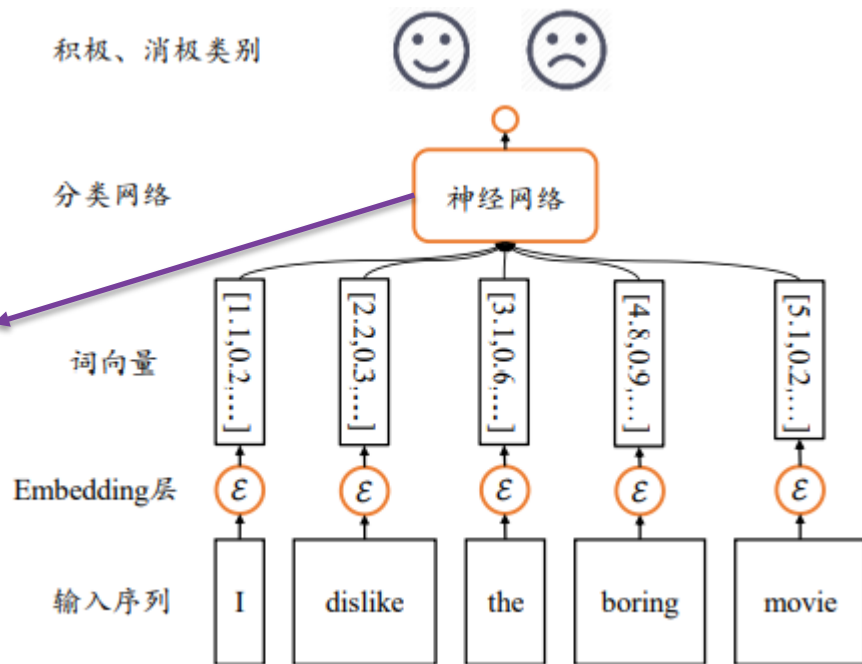
通过权值共享后，参数量大大减少，网络训练变得更加稳定高效。但是，这种网络结构并没有考虑序列之间的先后顺序，将词向量打乱次序仍然能获得相同的输出，无法获取有效的全局语义信息。

举个例子：
特朗普的儿子是谁？
谁的儿子是特朗普？

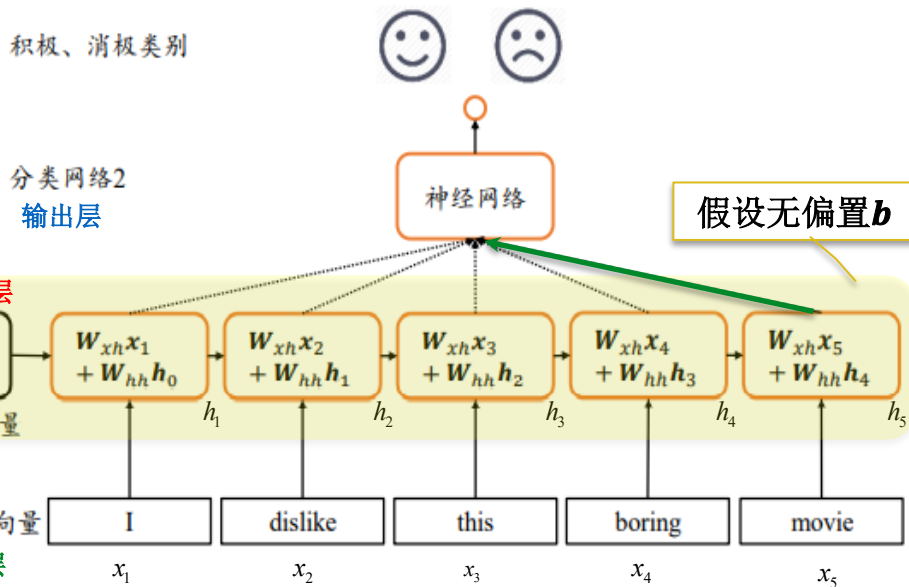


共享权值？

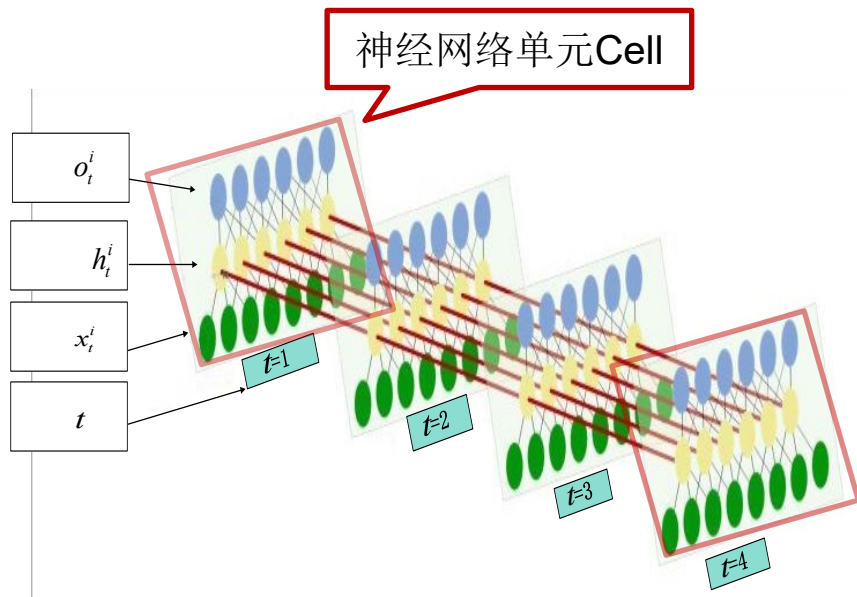
1. 网络参数量大；
2. 无法感知上下文
3. 无法处理变长输入



与卷积神经网络CNN一样，RNN也是参数共享，在时间维度上，**共享权重参数W**，能够感知上下文，能处理任意长度的时序数据



全局语义？



按时间线展开的循环神经网络

与卷积神经网络CNN一样，RNN也是参数共享，在时间维度上，**共享权重参数W**，能够感知上下文，能处理任意长度的时序数据

积极、消极类别



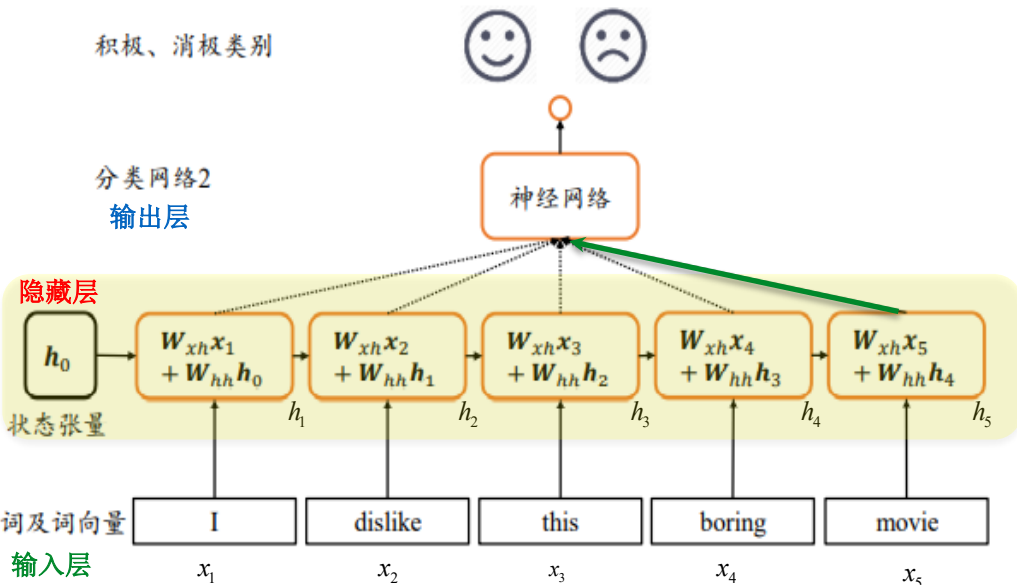
分类网络2
输出层

神经网络

隐藏层

状态张量

词及词向量
输入层

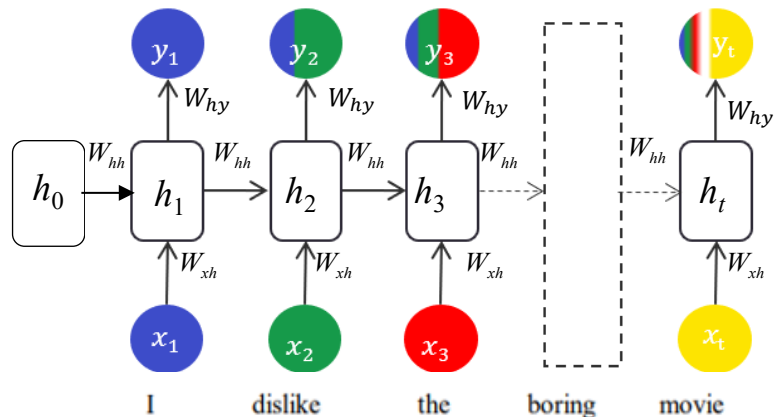


如果 x_t 词向量维度为10，用含1个隐层的RNN（隐状态 h_t 维度为20）实现情感分类（二分类），输入层、隐层和输出层的神经元数分别是多少？

$$h_t = f_W(h_{t-1}, x_t)$$

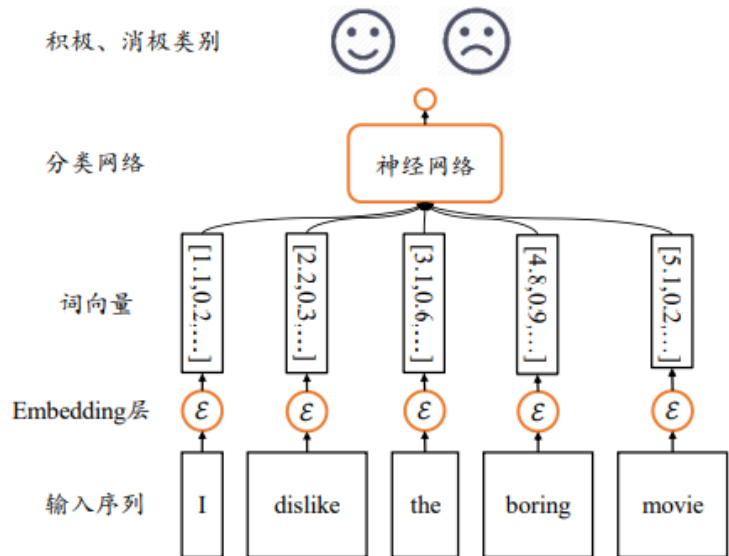
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$



h_t : 当前隐状态 (hidden state), h_{t-1} : 前一隐状态,
 x_t : 当前输入向量, f_W : 映射函数

$$h_t = \sigma(W_{xh}x_t + b_{xh} + W_{hh}h_{t-1} + b_{hh})$$



RNN的输入层、隐层和输出层的神经元数分别是取决于各自的维度。

#1.导入库

```
import torch, torch.nn as nn, torch.optim as optim
```

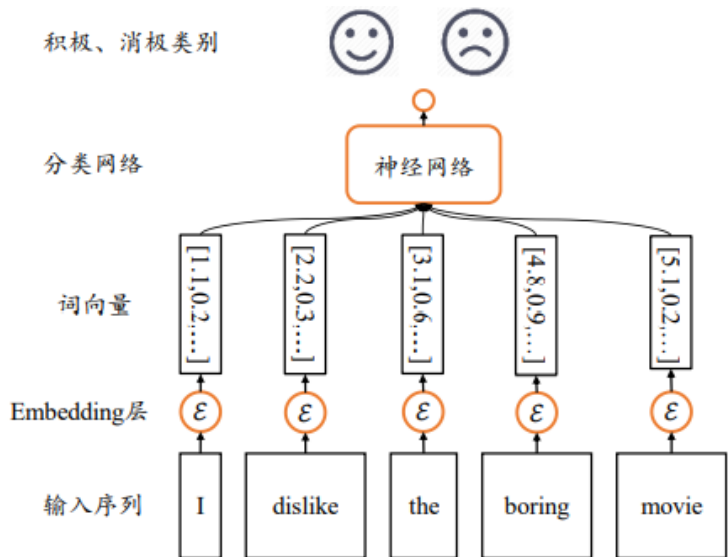
#2.构建一个简单的 RNN 模型

```
class RNNModel(nn.Module): # 定义一个继承自 nn.Module 的 RNNModel 类
    def __init__(self):      # 初始化函数，定义模型的层
        super().__init__()  # 调用父类 nn.Module 的初始化函数
        self.seq = nn.Sequential(
            nn.Embedding(10000, 128, padding_idx=0),
            nn.RNN(128, 64, batch_first=True))
        self.fc = nn.Sequential(
            nn.Linear(64, 1),
            nn.Sigmoid())      # 顺序容器Sequential，按顺序堆叠层
```

```
def forward(self, x):      # 前向传播函数，定义数据在模型中的流动路径
    out, _ = self.seq(x)   # out 的形状: (batch_size, sequence_length, hidden_size)
    return self.fc(out[:, -1]) # out[:, -1]提取每个样本最后一个时间步的输出
```

```
model = RNNModel()      #实例化模型
```

$$h_t = \sigma(W_{xh}x_t + b_{xh} + W_{hh}h_{t-1} + b_{hh})$$



RNN的输入层、隐层和输出层的神经元数分别是取决于各自的维度。

#3. 模型优化配置

```
optimizer = optim.Adam(model.parameters()) # 使用 Adam 优化器
criterion = nn.BCELoss() # 二元交叉熵损失
```

#4. 训练模型

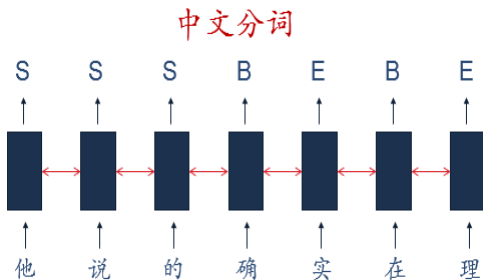
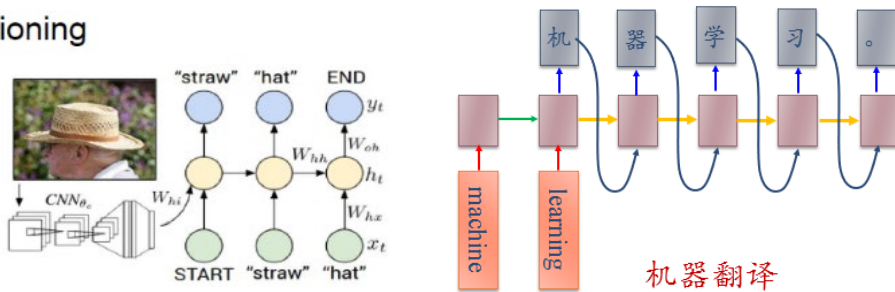
```
for _ in range(10):
    model.train() # 设置为训练模式
    optimizer.zero_grad() # 清零优化器中的梯度
    loss = criterion(model(X_train).squeeze(), y_train.float())
    loss.backward() # 反向传播，计算梯度
    optimizer.step() # 根据计算出的梯度更新模型参数
```

#5. 评估模型

```
model.eval() # 设置为评估模式
with torch.no_grad(): # 使用 torch.no_grad() 上下文管理器，在评估时关闭梯度计算
    out = model(X_test).squeeze() # out 的形状是 (test_batch_size,)
    accuracy = ((out>0.5)==y_test).float().mean().item() # 计算准确率
print(f"Test Accuracy: {accuracy}")
```

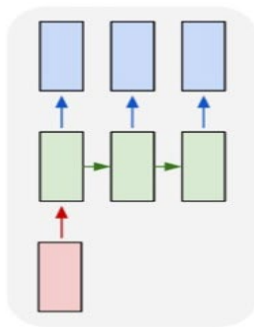
循环神经网络的应用

Image Captioning

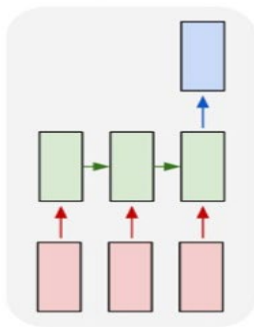


RNN分类:

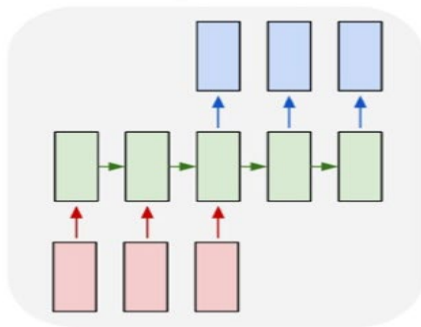
一对多



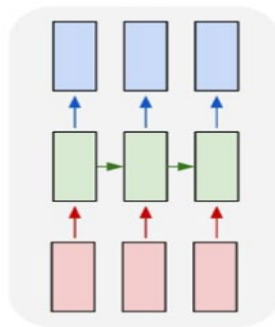
多对一



多对多



多对多



白鹭窥鱼立,
Egrets stood, peeping fishes.
青山照水开.
Water was still, reflecting mountains.
夜来风不动,
The wind went down by nightfall,
明月见楼台.
as the moon came up by the tower.

作诗

- 1) 输入一个输出多个, 例如输入一张图像, 输出这个图像的描述信息; 还有写诗、作曲、写文章等
- 2) 输入是多个, 输出则是一个, 例如输入段话, 输出这段话的情感; 还有股票预测、天气预报等
- 3) 输入是多个, 输出也是多个, 如机器翻译, 输入一段话, 输出也是一段话 (多个词); 还有自动文摘、多步股票预测等。
- 4) 多个输入和输出是同步的, 例如进行命名实体识别、词性分析、中文分词等

RNN的运行示例——CharRNN, Andrej karpathy, ICLR2016

网络参数怎么调整，尤其隐层参数怎么调整？

RNN的前向传播案例

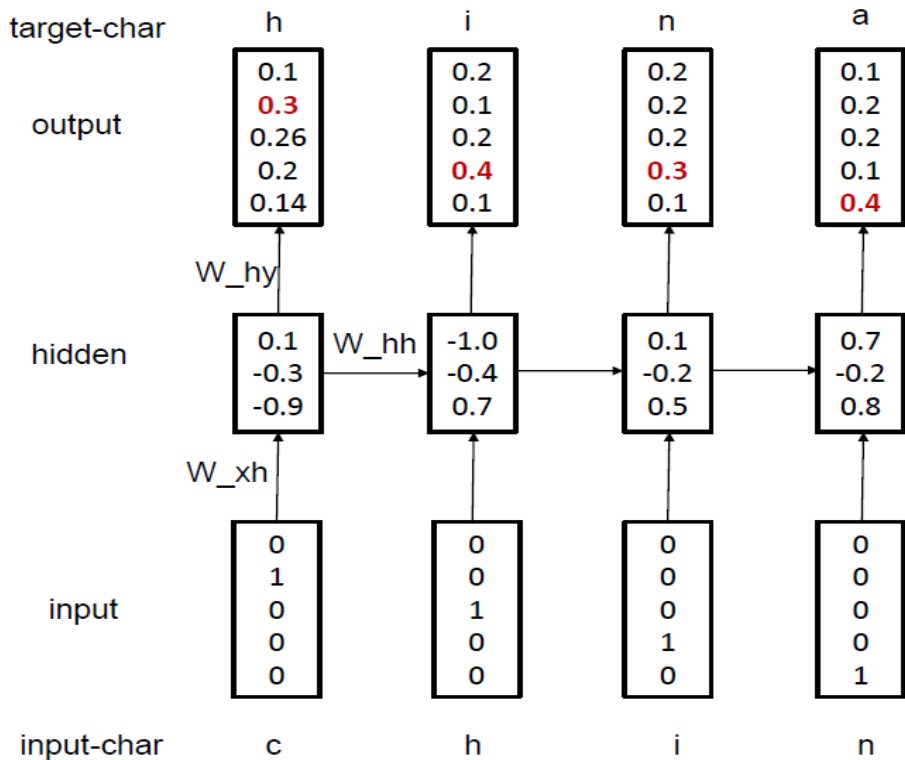
$$h_t = \tanh(w_{xh}x_t + w_{hh}h_{t-1})$$

$$\hat{y}_t = \text{softmax}(w_{hy}h_t)$$

词向量空间：

1	0	0	0	0
0	1	0	0	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	1
a	c	h	l	n

one-hot
编码



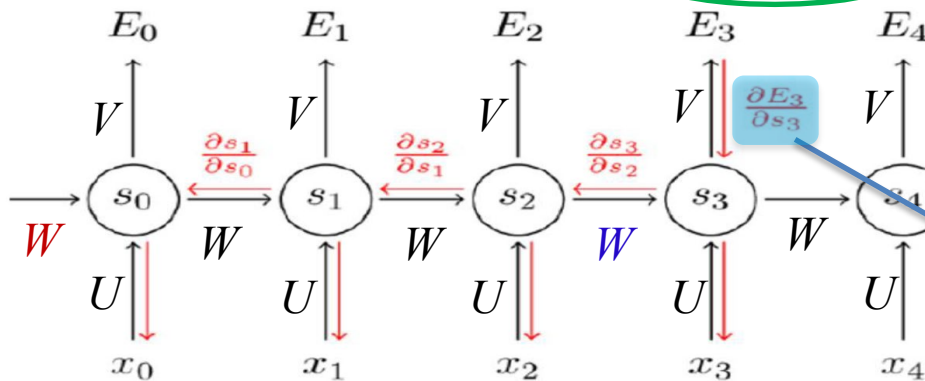
对于每个隐藏层状态的输出，都接一个全连接层预测下一个字母

Char RNN可用来写诗，写歌，生成文章，生成代码等

RNN的训练

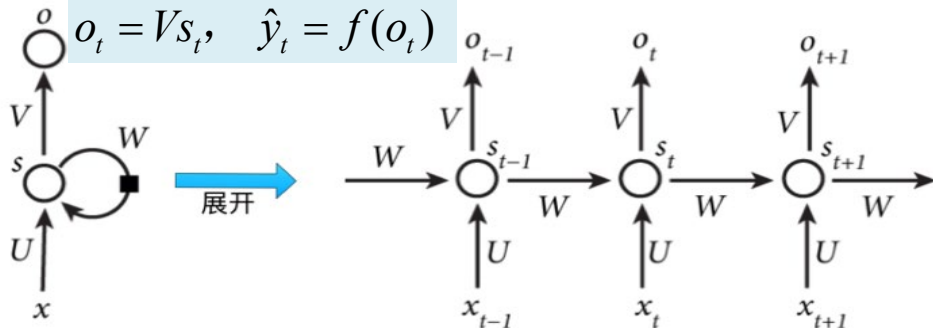
网络参数怎么调整，尤其
隐层参数怎么调整？

时序反向传播算法 (BPTT, Werbos, 1990)



$$s_t = \tanh(Ws_{t-1} + Ux_t)$$

$$o_t = Vs_t, \quad \hat{y}_t = f(o_t)$$



- 在每个时间步 $t = 0, 1, 2, 3, 4$ 神经网络的输出都会产生误差值: E_0, E_1, E_2, E_3, E_4 。
- 与前馈神经网络类似，RNN也使用反向传播梯度下降法更新权重(要计算误差对应的参数 U 、 V 和 W 的梯度)，例如

$$\frac{\partial E}{\partial W} = \sum_t \frac{\partial E_t}{\partial W}$$

➤ 以 E_3 为例,
$$\frac{\partial E_3}{\partial W} = \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial o_3} \frac{\partial o_3}{\partial s_3} \frac{\partial s_3}{\partial W}$$

$$\therefore \frac{\partial E_3}{\partial W} = \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial o_3} \frac{\partial o_3}{\partial s_3} \frac{\partial s_3}{\partial s_2} \frac{\partial s_2}{\partial s_1} \frac{\partial s_1}{\partial s_0} \frac{\partial s_0}{\partial W}$$

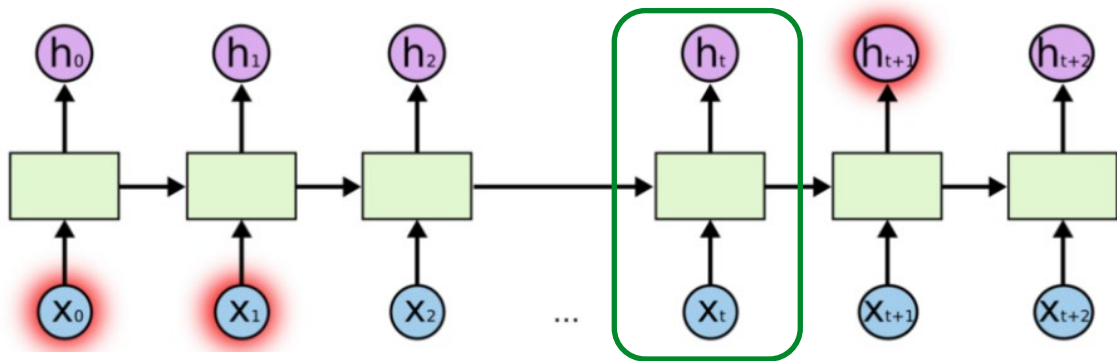
对于梯度消失，可以换激活函数、改变 W 的初始化、增加BN层，增大学习率、减少网络深度、添加 **Skip Connection** 等一系列的措施抑制，还可采用RNN的变体——LSTM等

包含了 $\text{diag}(\tanh'())W$ 的连乘运算，易出现梯度消失或梯度爆炸问题

RNN在处理较长的句子时，往往只能理解有限长度内的信息，而对于位于较长范围类的有用信息往往不能够很好的利用起来——**短时记忆**。

很久以前的输入，对当前时刻的网络影响较小；反向传播的梯度，也很难影响很久以前的输入
例如：

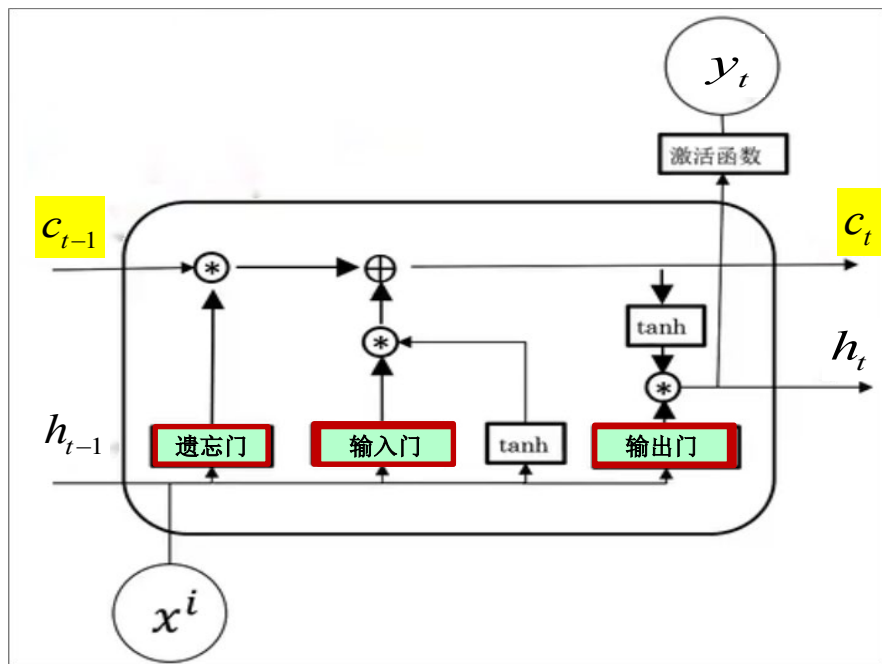
“今天天气太**美好**了，尽管路上发生了一件不愉快的事情，...，我马上调整好状态，**开开心心**地准备迎接美好的一天”



$$h_t = \sigma(W_{hh}h_{t-1} + b_{hh} + W_{xh}x_t + b_{xh})$$
$$= \tanh(W[h_{t-1}, x_t] + b_h)$$

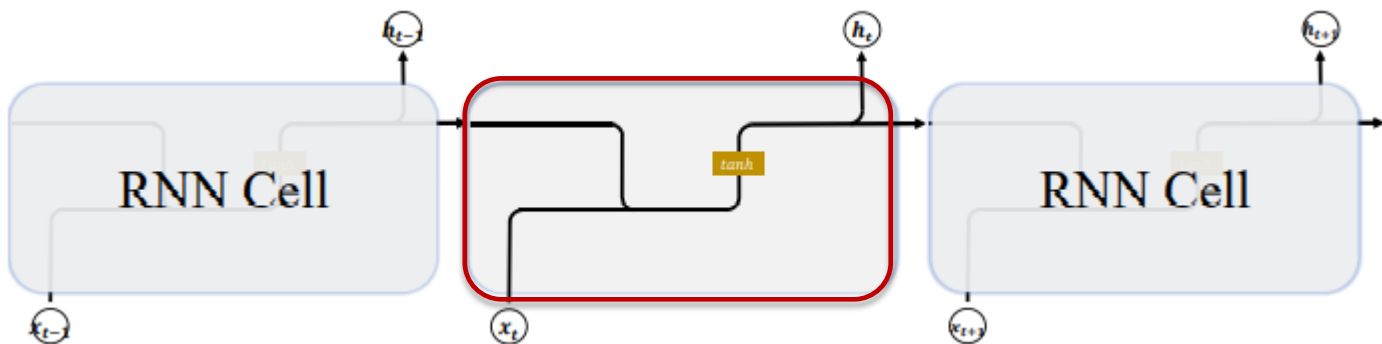
RNN缺陷：无法解决**长期依赖(Long Term Dependencies)**问题（当前系统的状态可能受
很长时间之前系统状态的影响）

1997 年，瑞士人工智能科学家 Jürgen Schmidhuber 提出了长短期记忆网络 (Long Short-Term Memory, 简称 LSTM)。



LSTM 新增了一个细胞状态 (cell state) C_t (也称为记忆单元, 记录了到当前时刻为止所有历史信息), 同时引入了门控(Gate)机制, 通过门控单元来控制信息的遗忘和刷新, 可以有效地解决长期依赖问题/梯度消失问题。

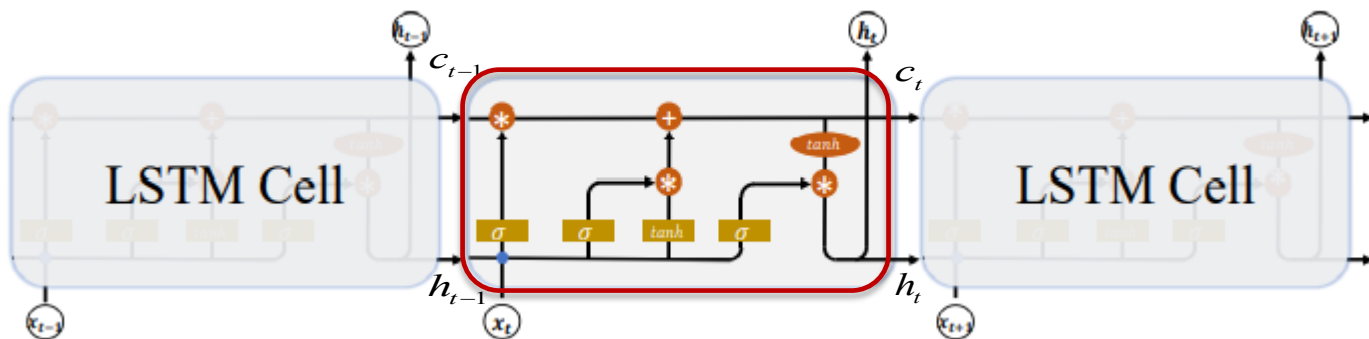
LSTM 相对于基础的 RNN 网络来说, 记忆能力更强, 更擅长处理较长的序列信号数据, **LSTM** 提出后, 被广泛应用于序列预测、自然语言处理等任务中, 几乎取代了基础的 RNN 模型。



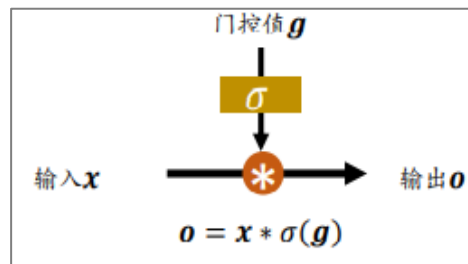
隐藏状态 h_{t-1} 与当前时间步的输入 x_t 经过线性变换后，通过激活函数 $\tanh$ 后得到下一时刻隐藏状态 h_t

基础RNN结构框图

$$h_t = \sigma(W_{xh}x_t + b_{xh} + W_{hh}h_{t-1} + b_{hh})$$



LSTM结构框图



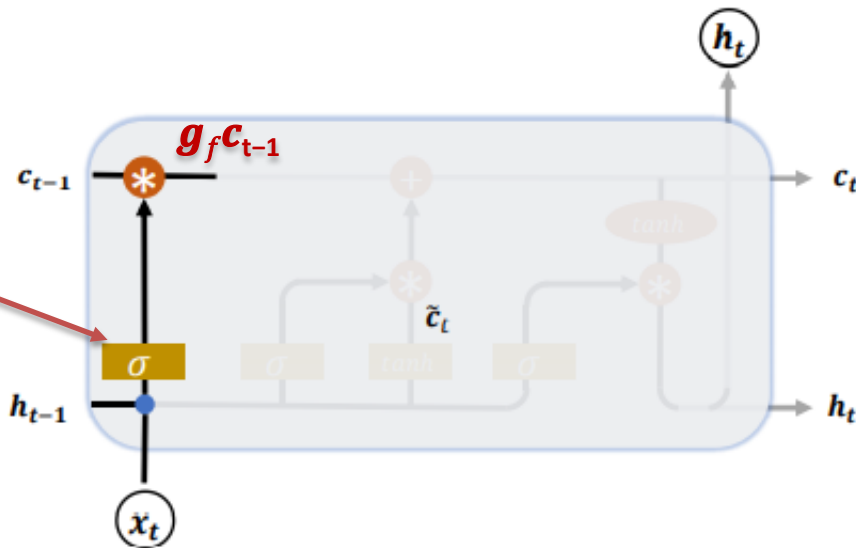
LSTM 新增了一个**细胞状态**(cell state) c_t (也称为**记忆单元**, 记录了到当前时刻为止所有历史信息), 同时引入了**门控**(Gate)机制, 通过门控单元来控制信息的遗忘和刷新

遗忘门

遗忘门的控制变量：

$$g_f = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

W_f 和 b_f 为遗忘门的参数，可由反向传播算法自动优化。 σ 一般使用 Sigmoid 函数。



当门控 $g_f=1$ 时，遗忘门全部打开，LSTM Cell接受上一个记忆状态 c_{t-1} 的所有信息；

当门控 $g_f=0$ 时，遗忘门关闭，LSTM Cell直接忽略 c_{t-1} ，输出为0的向量。

经过遗忘门后，LSTM Cell的状态向量变为 $g_f c_{t-1}$ 。

输入门

输入门用于控制 LSTM Cell对输入接收程度。

新的候选状态: $\hat{c}_t = \tanh(W_c [h_{t-1}, x_t] + b_c)$

输入门的控制变量:

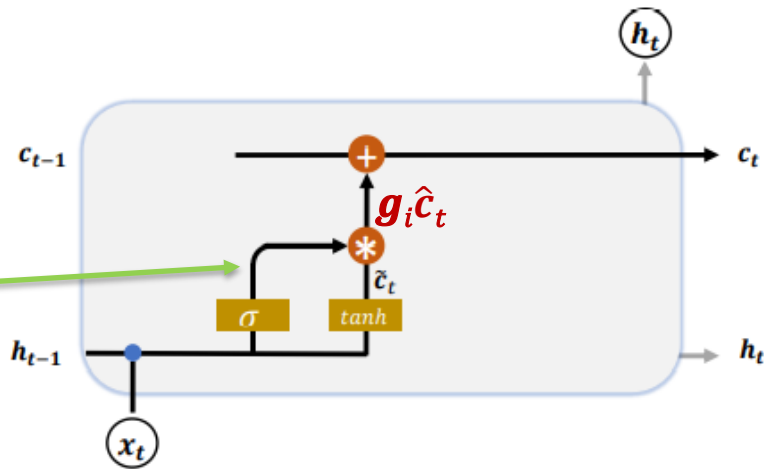
$$g_i = \sigma(W_i [h_{t-1}, x_t] + b_i)$$

其中 σ 一般使用Sigmoid函数, W_i 和 b_i 为输入门的参数, 通过反向传播算法优化(包括 W_c 和 b_c)。

当 $g_i = 0$ 时, LSTM Cell不接受任何的新候选状态 $\hat{c}_t$;

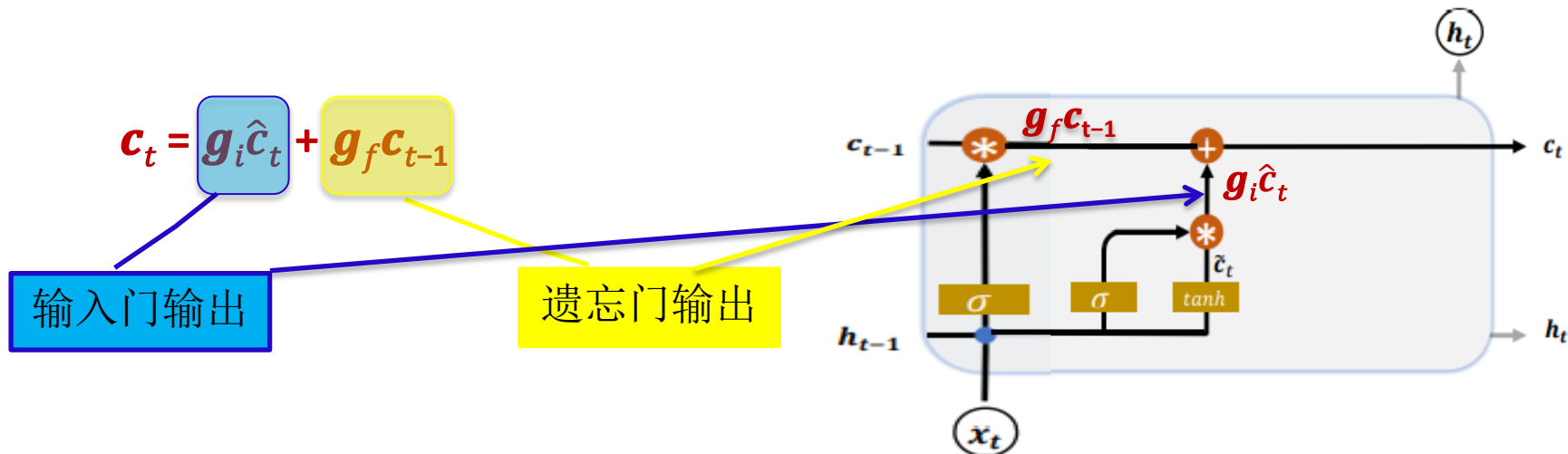
当 $g_i = 1$ 时, LSTM Cell全部接受新候选状态 $\hat{c}_t$ 。

经过输入门后, 待写入的状态为 $g_i \hat{c}_t$ 。



选择记忆/状态更新

在遗忘门和输入门的控制下，LSTM Cell 有选择地读取了上一个时间步的记忆 $\mathbf{c}_{t-1}$ 和当前时间步的新候选输入 $\hat{\mathbf{c}}_t$ ，当前记忆状态 $\mathbf{c}_t$ 的刷新方式为



输出门

输出门的门控变量: $g_o = \sigma(W_o[h_{t-1}, x_t] + b_o)$

其中 W_o 和 b_o 为输出门的参数，通过反向传播算法自动优化， σ 一般使用 Sigmoid 函数。

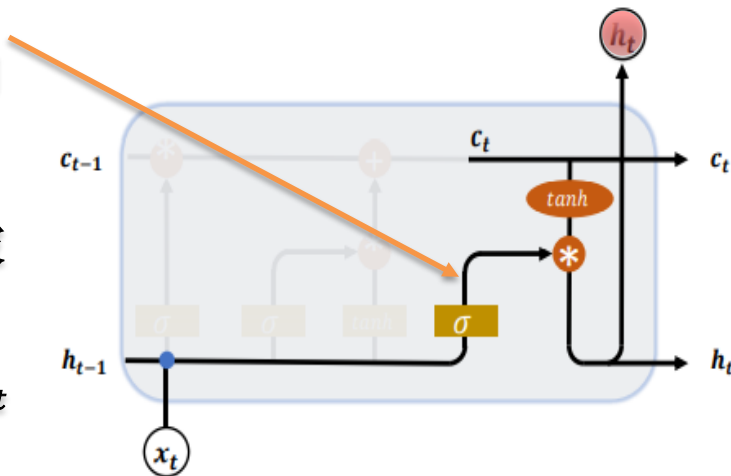
当 $g_o = 0$ 时，输出关闭，LSTM Cell的内部记忆完全被隔断，输出为 0 的向量；

当 $g_o = 1$ 时，输出完全打开，LSTM Cell的状态向量 c_t 全部用于输出。

LSTM Cell的输出:

$$h_t = g_o \cdot \tanh(c_t)$$

$g_o \in [0,1]$, $\tanh(c_t) \in [-1,1]$, 输出 $h_t \in [-1,1]$



在 LSTM 内部，细胞状态（cell state） c_t 并不会全部输出，而是在输出门的作用下有选择地输出

遗忘门: $g_f = \text{sigmoid}(W_f [h_{t-1}, x_t] + b_f)$

输入门: $g_i = \text{sigmoid}(W_i [h_{t-1}, x_t] + b_i)$

输出门: $g_o = \text{sigmoid}(W_o [h_{t-1}, x_t] + b_o)$

候选记忆状态: $\hat{c}_t = \tanh(W_c [h_{t-1}, x_t] + b_c)$

新记忆状态: $c_t = g_f c_{t-1} + g_i \hat{c}_t$

输出隐状态: $h_t = g_o \cdot \tanh(c_t)$

LSTM Cell 参数量是RNN Cell参数的4倍

遗忘门

决定丢弃多少旧信息

输入门

判断要保留多少当前输入

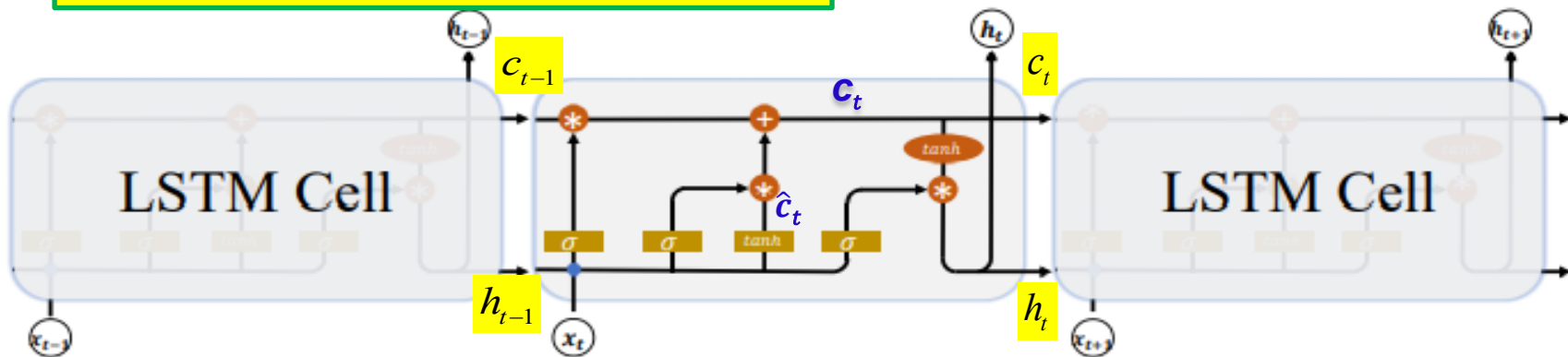
状态
更新

根据输入门和遗忘门的控制，
更新记忆状态

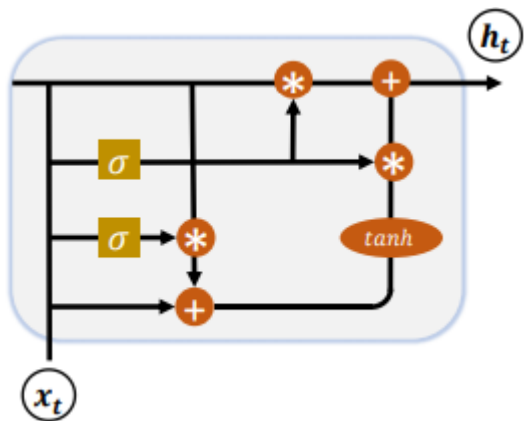
输出门

选择重要的信息输出

$$\begin{aligned} \text{RNN: } h_t &= \sigma(W_{hh} h_{t-1} + b_{hh} + W_{xh} x_t + b_{xh}) \\ &= \tanh(W[h_{t-1}, x_t] + b_h) \end{aligned}$$



门控循环单元(GRU , Gated Recurrent Unit, 2014)本质上就是一个没有输出门的长短期记忆网络



- **LSTM** 具有更长的记忆能力，在大部分序列任务上面都取得了比基础RNN 模型更好的性能表现，更重要的是，**LSTM** 不容易出现梯度消失现象。
- 但是 **LSTM** 结构相对较复杂，因多个门控状态的引入，计算代价较高，模型参数量较大。

GRU 把内部状态 C_t 和隐藏状态 h_t 合并，统一为隐藏状态 h_t ，门控数量也减少到 2 个：**复位门/重置门** 和**更新门**

重置门

重置门用于控制上一时间步的隐状态 h_{t-1} 进入 GRU 的量

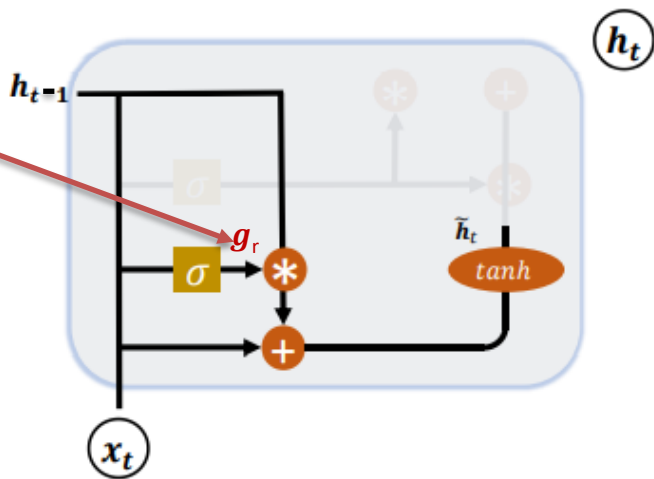
门控变量: $\mathbf{g}_r = \sigma(\mathbf{W}_r[h_{t-1}, \mathbf{x}_t] + \mathbf{b}_r)$

其中 $\mathbf{W}_r$ 和 $\mathbf{b}_r$ 为复位门的参数, 通过反向传播算法自动优化, σ 一般使用 Sigmoid 函数。

$$\hat{\mathbf{h}}_t = \tanh(\mathbf{W}_h [\mathbf{g}_r h_{t-1}, \mathbf{x}_t] + \mathbf{b}_h)$$

当 $\mathbf{g}_r = 0$ 时, 新候选状态 $\hat{\mathbf{h}}_t$ 全部来自于输入 $\mathbf{x}_t$, 不接受上一时间步的隐藏状态 h_{t-1} , 此时相当于复位 h_{t-1} 。

当 $\mathbf{g}_r = 1$ 时, h_{t-1} 和输入 $\mathbf{x}_t$ 共同产生新候选状态 $\hat{\mathbf{h}}_t$ 。



更新门

更新门用控制上一时间步隐藏状态 h_{t-1} 和**新候选状态** $\hat{h}_t$ 对当前时间步隐藏状态 h_t 的影响程度。（**决定保留或放弃**）

门控变量： $g_z = \sigma(W_z[h_{t-1}, x_t] + b_z)$

其中 W_z 和 b_z 为输出门的参数，通过反向传播算法自动优化， σ 一般使用 Sigmoid 函数。

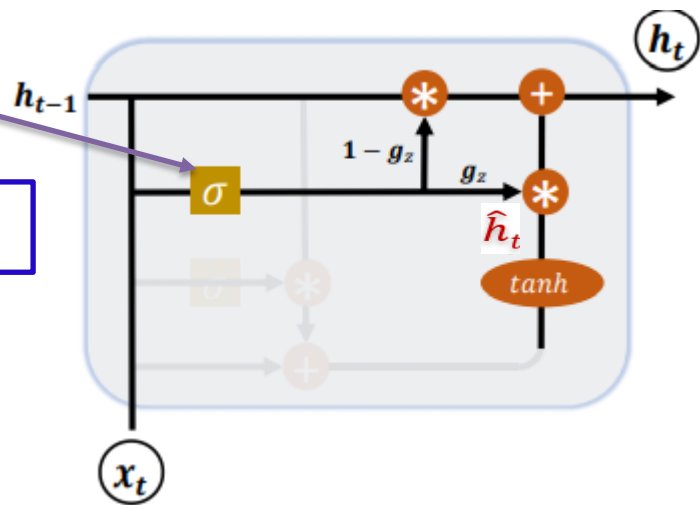
保留哪些旧状态

接收哪些新状态

$$h_t = (1 - g_z)h_{t-1} + g_z \hat{h}_t$$

当 $g_z = 0$ 时， h_t 全部来自上一时间步隐藏状态 h_{t-1} ；

当 $g_z = 1$ 时， h_t 全部来自新状态 $\hat{h}_t$ 。



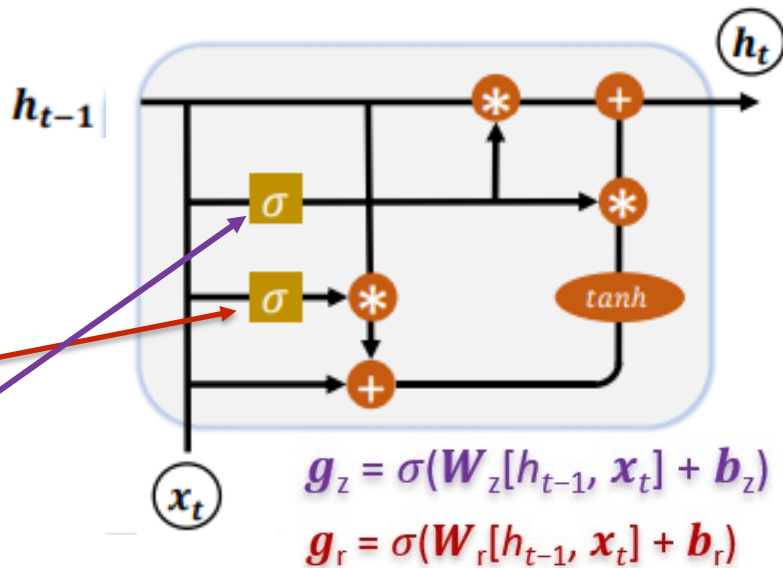
门控循环单元(GRU, Gated Recurrent Unit, 2014)本质上就是一个没有输出门的长短期记忆网络

GRU 把内部状态 C_t 和隐状态 h_t 合并, 统一为隐状态 h_t , 门控数量也减少到 2 个: **重置门**和**更新门**

✓ **重置门**/复位门: 控制忽略前一时刻的状态信息的程度;

✓ **更新门** g_z : 遗忘门和输入门的结合体, 控制前一时刻的状态信息被带入到当前状态中的程度

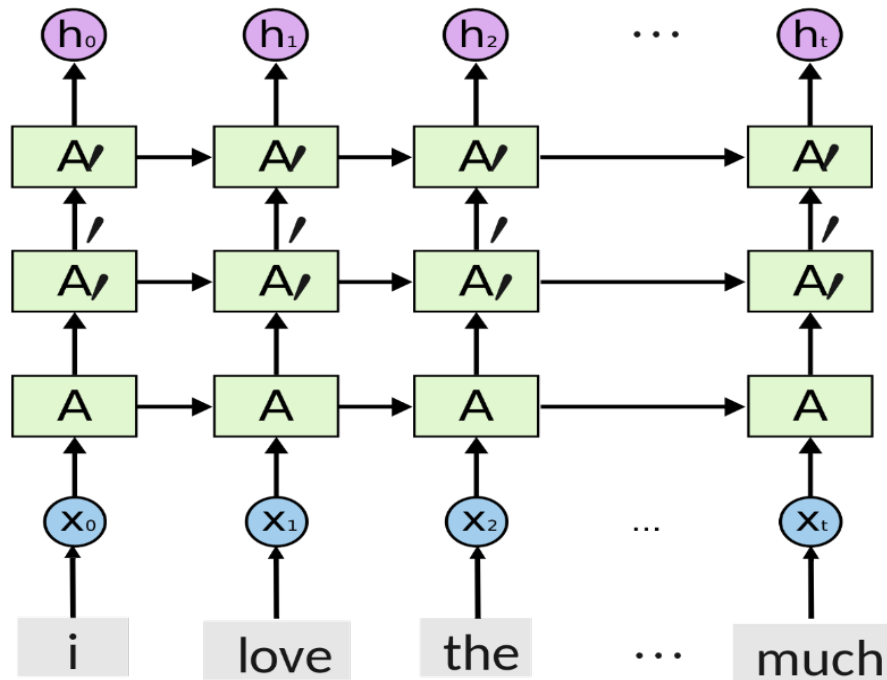
Cell参数量: GRU 是LSTM 的3/4



新候选状态: $\hat{h}_t = \tanh(W_h [g_r h_{t-1}, x_t] + b_h)$

隐藏状态更新: $h_t = (1 - g_z)h_{t-1} + g_z \hat{h}_t$

StackRNN（堆叠循环神经网络）：多层 RNN



1) 多层 RNN 可以让 RNN 网络得到词语序列更加复杂的表示。

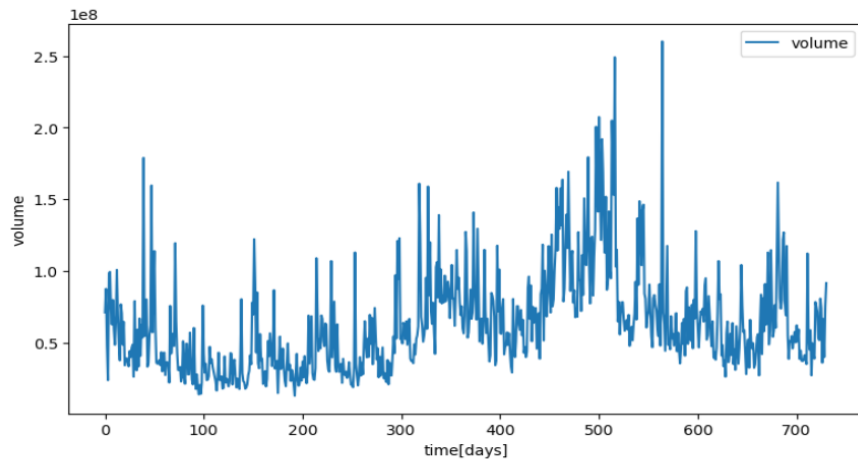
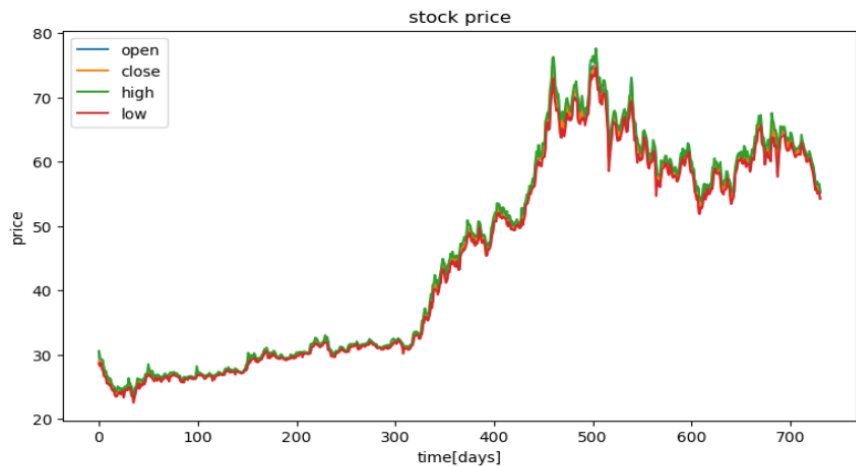
2) 下面的 RNN 层可以得到低阶特征。

3) 上面的 RNN 层可以得到高阶特征。

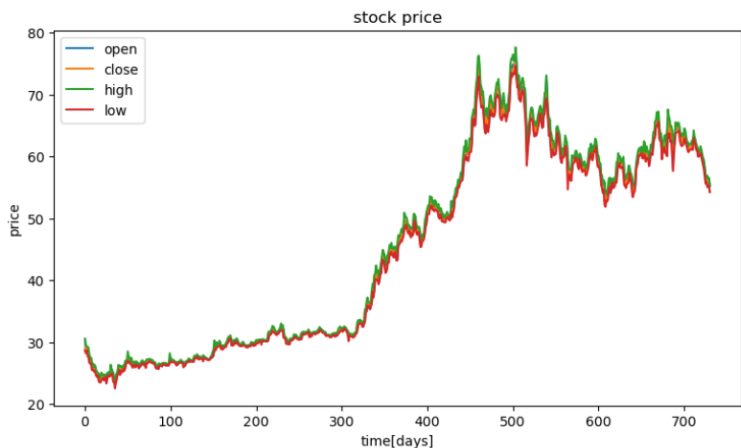
指标数据：

- 开盘
- 收盘
- 最低
- 最高
- 成交量

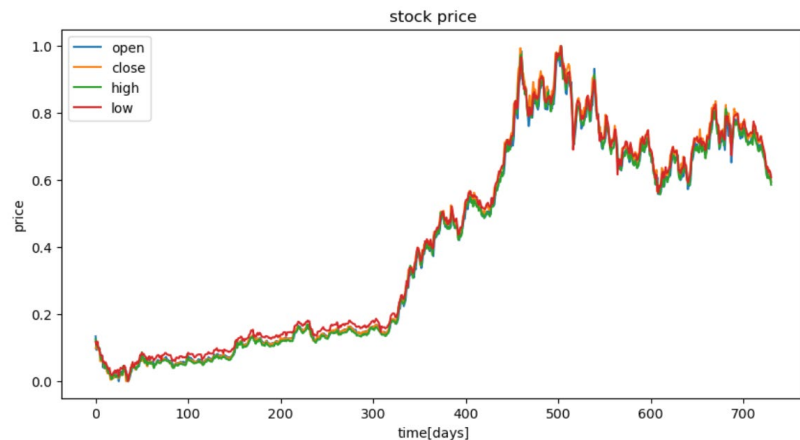
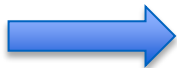
	date	open	high	low	close	volume
0	2016-01-04	30.57	30.57	28.63	28.78	70997200
1	2016-01-05	28.41	29.54	28.23	29.23	87498504
2	2016-01-06	29.03	29.39	28.73	29.26	48012112
3	2016-01-07	28.73	29.25	27.73	28.50	23647604
4	2016-01-08	28.73	29.18	27.63	28.67	98239664



```
# min-max 归一化
from sklearn.preprocessing import MinMaxScaler
min_max_scaler = MinMaxScaler()
df['open'] = min_max_scaler.fit_transform(df['open'].values.reshape(-1,1))
df['high'] = min_max_scaler.fit_transform(df['high'].values.reshape(-1,1))
df['low'] = min_max_scaler.fit_transform(df['low'].values.reshape(-1,1))
df['close'] = min_max_scaler.fit_transform(df['close'].values.reshape(-1,1))
```

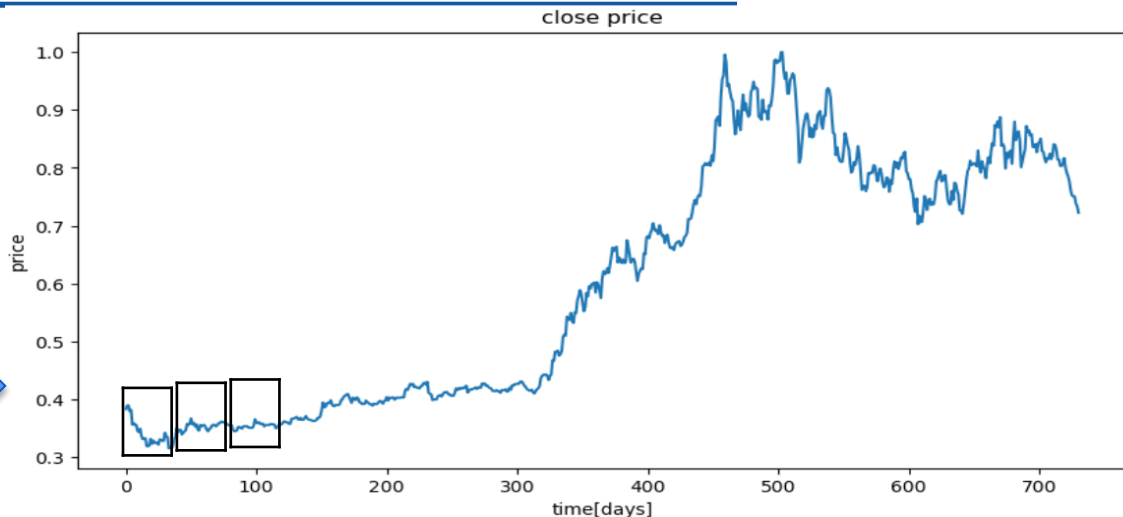
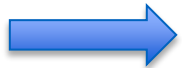


数据预处理 (归一化)
之后可视化结果:



股票走势转化为
回归问题进行预测

加窗 (window)



样本结果标记：窗口后涨跌作为样本标记,即窗口后的股票价格（例如**收盘价**）作为预测结果

回归模型采用MSE作为损失函数，对预测结果进行评价

20天为一个周期序列，训练集：577个样本，验证集：73个样本，测试集：73个样本

----- 构建2层LSTM模型 -----

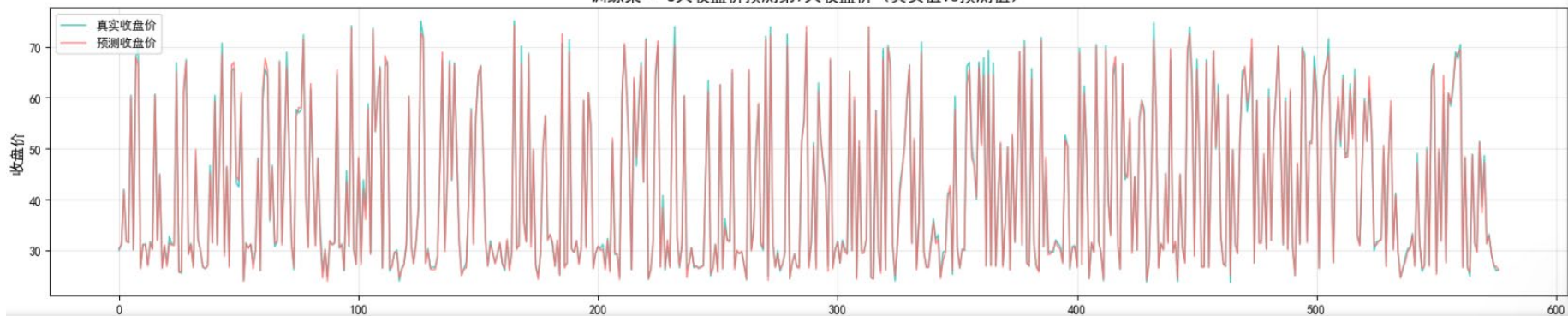
```
class StockLSTM(nn.Module):          # 定义一个继承自 nn.Module 的 StockLSTM 类
    def __init__(self, input_size=5, hidden_size = 64, num_layers = 2):      # 初始化函数
        super().__init__()          # 调用父类的初始化函数
        self.lstm = nn.LSTM(         # 定义 LSTM 层
            input_size = input_size, # 输入特征数: 5 (开、收、低、高、量)
            hidden_size = hidden_size, # 隐藏层神经元数
            num_layers = num_layers,   # 2层LSTM
            batch_first = True         # 输入格式: [batch, seq_len, input_size] )
        self.fc = nn.Linear(hidden_size, 1) # 定义输出层: 预测收盘价

    def forward(self, x):             # 前向传播函数
        _, (hn, _) = self.lstm(x)
        return self.fc(hn[-1])       # 取最后一层最后一个时间步的隐藏状态做预测

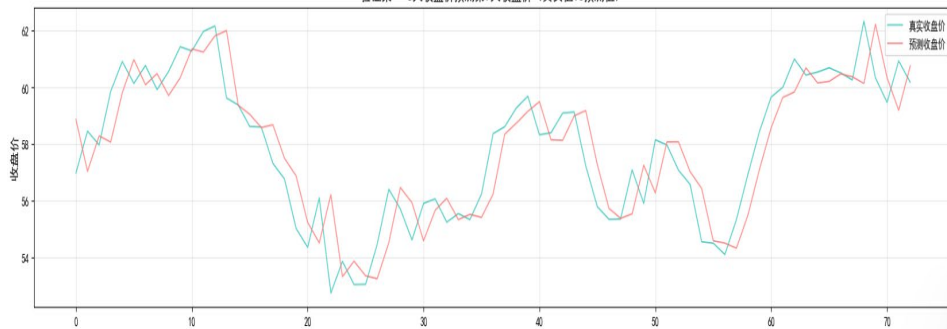
# 初始化模型、损失函数、优化器
model = StockLSTM().to(device)
criterion = nn.MSELoss()          # 回归任务用MSE损失
optimizer = optim.Adam(model.parameters(), lr = 1e-3)
epochs = 200                      # 训练轮次
```

```
train_pred, train_true = get_pred_true(train_loader, model, scaler)  
val_pred, val_true = get_pred_true(val_loader, model, scaler)  
test_pred, test_true = get_pred_true(test_loader, model, scaler)
```

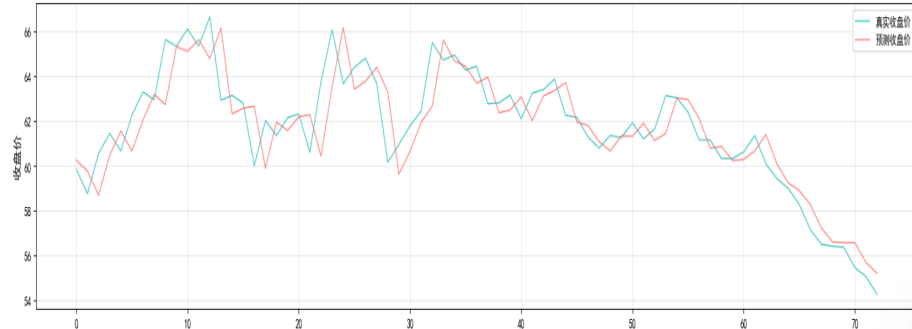
训练集 - 8天收盘价预测第9天收盘价 (真实值vs预测值)



验证集 - 8天收盘价预测第9天收盘价 (真实值vs预测值)



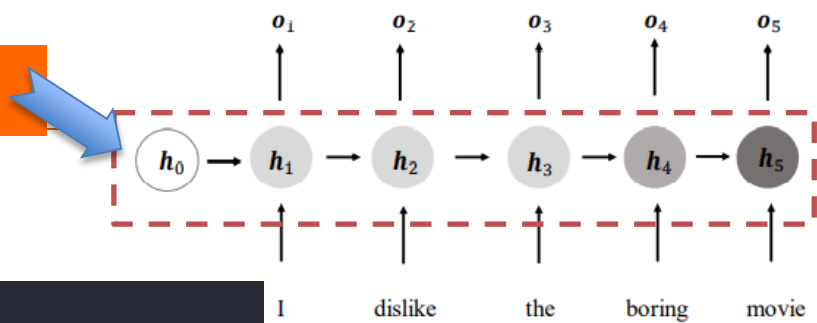
测试集 - 8天收盘价预测第9天收盘价 (真实值vs预测值)





1. 了解序列数据的特点，实际场景中有哪些主要的序列任务？
2. 有哪些常用的循环神经网络（RNN）模型？了解RNN与卷积神经网络CNN、BP神经网络之间的区别。为什么说RNN具有记忆功能？
3. 什么是词向量（词嵌入，Word Embedding）？词向量的特性是什么？有哪些预训练的语言模型？为什么说glove、word2vec和fasttext无法解决多义词问题？
4. 针对具体的情感分类问题，能设计RNN的结构模型。
5. RNN的应用模式可以分为哪几类？并给出应用举例说明。
6. 为什么说简单的RNN无法解决长期依赖问题？LSTM是如何有效地解决长期依赖问题的？
7. 对比分析简单的RNN、LSTM和GRU的参数数量。和LSTM相比，GRU有什么特点？

RNN实现 (Pytorch) —— nn.RNN、nn.RNNCell



```
rnn = nn.RNN(input_size=100, hidden_size=20, num_layers=4)
print(rnn)
x = torch.randn(10, 3, 100)
out, h = rnn(x)
print(out.shape, h.shape)
```

3个句子，每个句子10个单词，词向量维度为100

```
RNN(100, 20, num_layers=4)
torch.Size([10, 3, 20]) torch.Size([4, 3, 20])
```

```
1 cell1 = nn.RNNCell(100, 20)
2 h1 = torch.zeros(3, 20)
3 for xt in x:
4     h1 = cell1(xt, h1)
5 print(h1.shape)
6 torch.Size([3, 20])
```

$$h_t = \sigma(W_{xh}x_t + b_{xh} + W_{hh}h_{t-1} + b_{hh})$$

`ht = rnncell(xt, ht_1)`

- `xt`: [b, word vec]
- `ht_1/ht`: [b, h dim]
- `out = torch.stack([h1, h2, ..., ht])`

`out, ht = RNN(x, h0)`

- `x`: [seq len, b, word vec]
- `h0/ht`: [num layers, b, h dim]
- `out`: [seq len, b, h dim]

D*num layers (单向, D=1; 双向, D=2)
D*h dim (单向, D=1; 双向, D=2)

```
1 cell1 = nn.RNNCell(100, 30)
2 cell2 = nn.RNNCell(30, 20)
3 h1 = torch.zeros(3, 30)
4 h2 = torch.zeros(3, 20)
5 for xt in x:
6     h1 = cell1(xt, h1)
7     h2 = cell2(h1, h2)
8
9 print(h2.shape)
10 torch.Size([3, 20])
```

input_size: 输入特征x的维度
hidden_size: 隐状态h的维度
nonlinearity: 激活函数('tanh' or 'relu').
Default: 'tanh'
bias: 是否使用偏置(`b_xh` and `b_hh`).
Default: True

nn.RNNCell方式实现RNN，比较灵活，每一层的隐状态维度可以不同

LSTM 实现 (Pytorch) —— nn.LSTM、nn.LSTMCell

```
1 lstm = nn.LSTM(input_size=100, hidden_size=20, num_layers=4)
2 print(lstm)
3 x = torch.randn(10, 3, 100)
4 out, (h, c) = lstm(x)
5 print(out.shape, h.shape, c.shape)
6
7 torch.Size([10, 3, 20])
8 torch.Size([4, 3, 20])
9 torch.Size([4, 3, 20])
```

3个句子，每个句子10个单词，词向量长度100

D*num layers: 单向, D=1; 双向, D=2
D*h: 单向, D=1; 双向, D=2

```
1 print('two layer lstm')
2 cell1 = nn.LSTMCell(input_size=100, hidden_size=30)
3 cell2 = nn.LSTMCell(input_size=30, hidden_size=20)
4 h1 = torch.zeros(3, 30)
5 c1 = torch.zeros(3, 30)
6 h2 = torch.zeros(3, 20)
7 c2 = torch.zeros(3, 20)
8 for xt in x:
9     h1, c1 = cell1(xt, [h1, c1])
10    h2, c2 = cell2(h1, [h2, c2])
11 print(h2.shape, c2.shape)
12
13 torch.Size([3, 20])
14 torch.Size([3, 20])
```

ht, ct = lstmcell(xt, [ht_1, ct_1])

xt: [b, vec]

ht/ct: [b, h]

nn.LSTMCell方式实现LSTM，比较灵活，每一层的隐状态维度可以不同

out, (ht, ct) = lstm(x, [ht_1, ct_1])

x: [seq, b, vec]

h/c: [num_layer, b, h]

out: [seq, b, h]

input_size: x的维度
hidden_size: h的维度
num_layers: 默认 1
nonlinearity: 默认 'tanh'
bias: 默认 True
batch_first: 默认 False
dropout: 默认 0
bidirectional: 默认 False

TEXT.build_vocab(train_data, max_size=10000, vectors='glove.6B.100d')

class RNN(nn.Module):

```
def __init__(self, vocab_size, embedding_dim, hidden_dim):
    super(RNN, self).__init__()
    self.embedding = nn.Embedding(vocab_size, embedding_dim)
    self.rnn = nn.LSTM(embedding_dim, hidden_dim, num_layers=2,
                       bidirectional=False, dropout=0.5)
    self.fc = nn.Linear(hidden_dim * 1, 1)
    self.dropout = nn.Dropout(0.5)
```

单向LSTM
(2个隐层)

def forward(self, x):

embedding = self.dropout(self.embedding(x))

output, (hidden, cell) = self.rnn(embedding)

out = self.fc(output[-1, :, :]) # 因为有max_seq_len个时态，所以取最后一个时态即-1层
return out

rnn = RNN(len(TEXT.vocab), 100, 64)

双向LSTM (Pytorch)

```
TEXT.build_vocab(train_data, max_size=10000, vectors='glove.6B.100d')
```

2. 搭建循环网络

```
class RNN(nn.Module):
```

```
def __init__(self, vocab_size, embedding_dim, hidden_dim):  
    super(RNN, self).__init__()  
    self.embedding = nn.Embedding(vocab_size, embedding_dim)  
    self.rnn = nn.LSTM(embedding_dim, hidden_dim, num_layers=2,  
                        bidirectional=True, dropout=0.5)
```

```
self.fc = nn.Linear(hidden_dim * 2, 1)
```

```
self.dropout = nn.Dropout(0.5)
```

```
def forward(self, x):
```

```
    embedding = self.dropout(self.embedding(x))
```

```
    output, (hidden, cell) = self.rnn(embedding)
```

```
    hidden = torch.cat([hidden[-2], hidden[-1]], dim=1)
```

```
    hidden = self.dropout(hidden)
```

```
    out = self.fc(hidden)
```

```
    return out
```

初始化网络

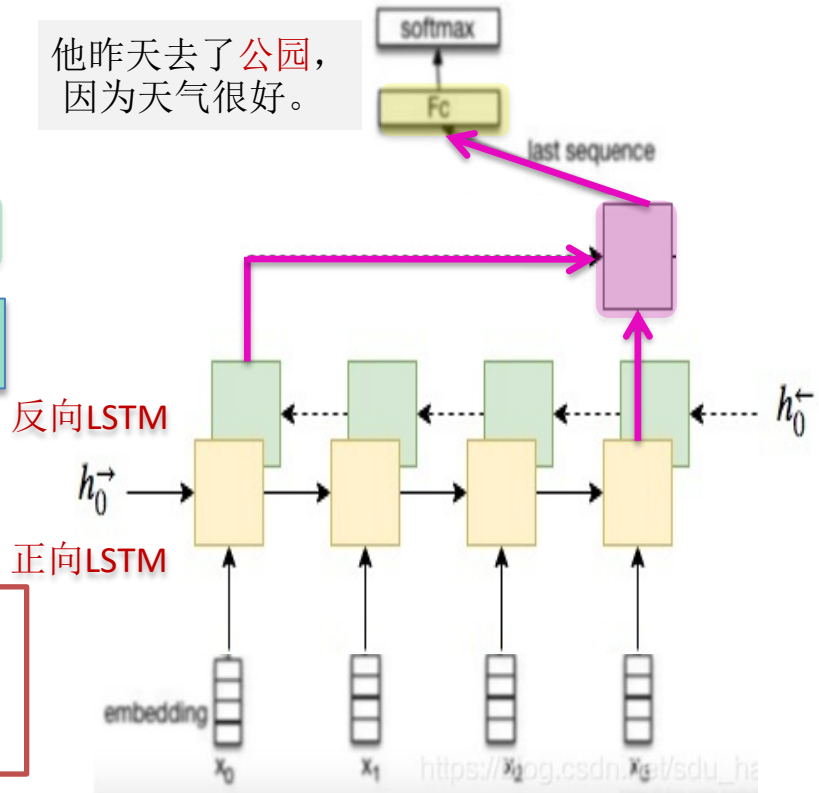
```
rnn = RNN(len(TEXT.vocab), 100, 64)
```

双向LSTM
(2个隐层)

- 正向LSTM的最后一个隐状态:
从句子的最后一个单词获得
- 反向LSTM的最后一个隐状态:
从句子的第一个单词获得

input_size: x的维度; **hidden_size**: h的维度
num_layers: 默认 1; **nonlinearity**: 默认'tanh'
bias: 默认True; **batch_first**: 默认 False
dropout: 默认 0; **bidirectional**: 默认False

他昨天去了公园,
因为天气很好。



单层双向LSTM的示意图

一般取前向和反向LSTM在最后一个时间步的隐藏状态, 进行拼接, 再接一个全连接层进行分类。

GRU 实现 (Pytorch) ——GRU层

nn.GRU、nn.GRUCell

```
TEXT.build_vocab(train_data, max_size=10000, vectors='glove.6B.100d')
```

```
class RNN(nn.Module):
```

```
    def __init__(self, vocab_size, embedding_dim, hidden_dim):
```

```
        super(RNN, self).__init__()
```

```
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
```

```
        self.rnn = nn.GRU(embedding_dim, hidden_dim, num_layers=2,  
                           bidirectional=False, dropout=0.5)
```

```
        self.fc = nn.Linear(hidden_dim * 1, 1)
```

```
        self.dropout = nn.Dropout(0.5)
```

单向GRU
(2个隐层)

```
    def forward(self, x):
```

```
        embedding = self.dropout(self.embedding(x))
```

```
        output, hidden = self.rnn(embedding)
```

```
        out = self.fc(output[-1, :, :]) #因为有max_seq_len个时态, 所以取最后一个时态即-1层
```

```
        return out
```

```
rnn = RNN(len(TEXT.vocab), 100, 64)
```

input_size: x的维度

hidden_size: h的维度

num_layers: 默认 1

nonlinearity: 默认'tanh'

bias: 默认True

batch_first: 默认 False

dropout: 默认 0

bidirectional: 默认False

out, ht = GRU(x, h0)

▪ x: [seq len, b, word vec]

▪ h0/ht: [num layers, b, h dim]

▪ out: [seq len, b, h dim]

D*num layers: 单向, D=1; 双向, D=2

D*h dim: 单向, D=1; 双向, D=2